

Image Representation and Filtering

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



DaSCI

Instituto Andaluz de Investigación en
Data Science and Computational Intelligence

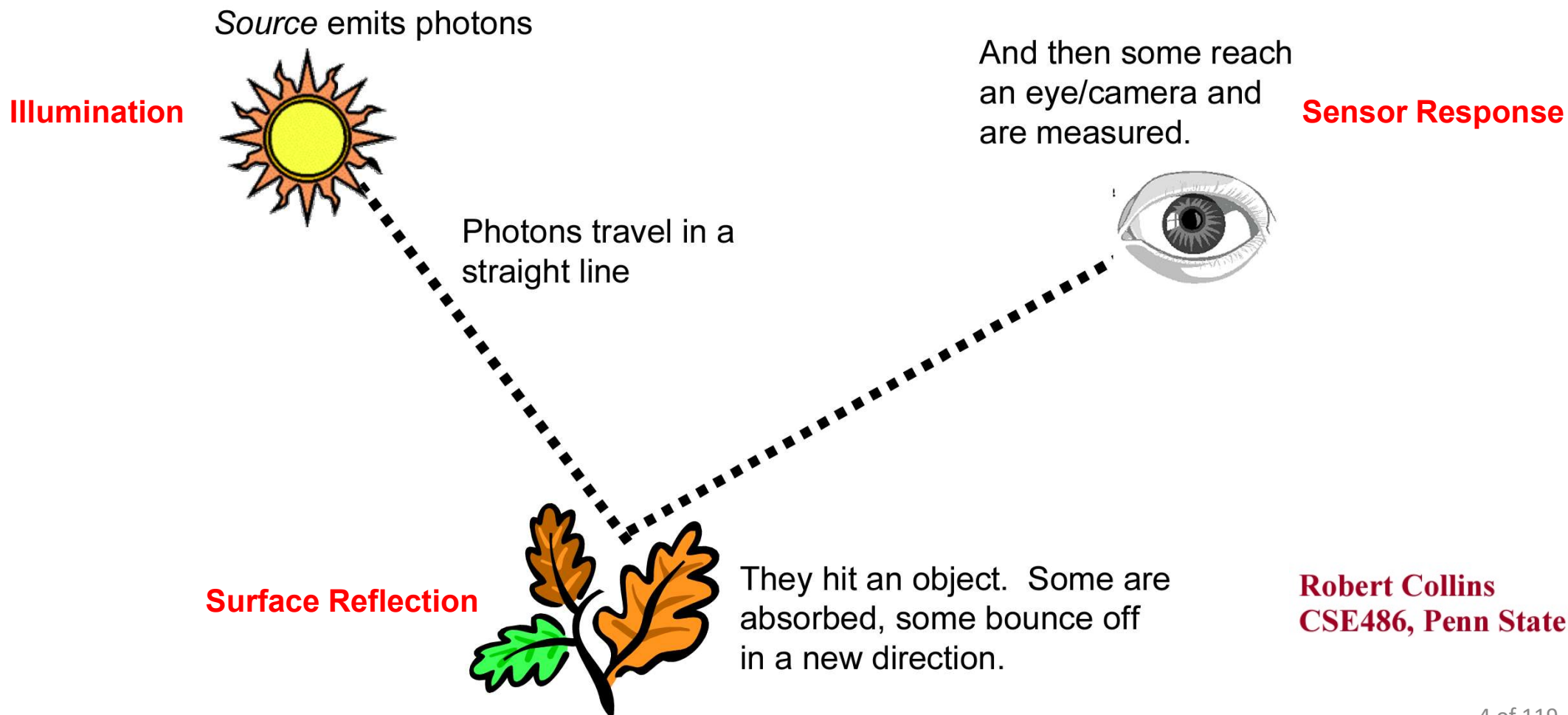
Index

- What is an image?
- Digital color images
- Introduction to linear filtering and noise reduction
- Cross-correlation and convolution
- Examples of linear filters
- Nonlinear filters, thresholding and histogram equalization

Readings

- ***Image Formation & Sampling***
 - Szeliski (2022), Chapter 2
- ***Image Filtering & Edge Detection***
 - Szeliski (2022), Chapter 3 & 7.2
 - Forsyth and Ponce (2012), Chapter 4, 5.1 & 5.2
- Slides credit: many slides have been adapted from those by Noah Snavely, Derek Hoiem and Stan Birchfield, among others.

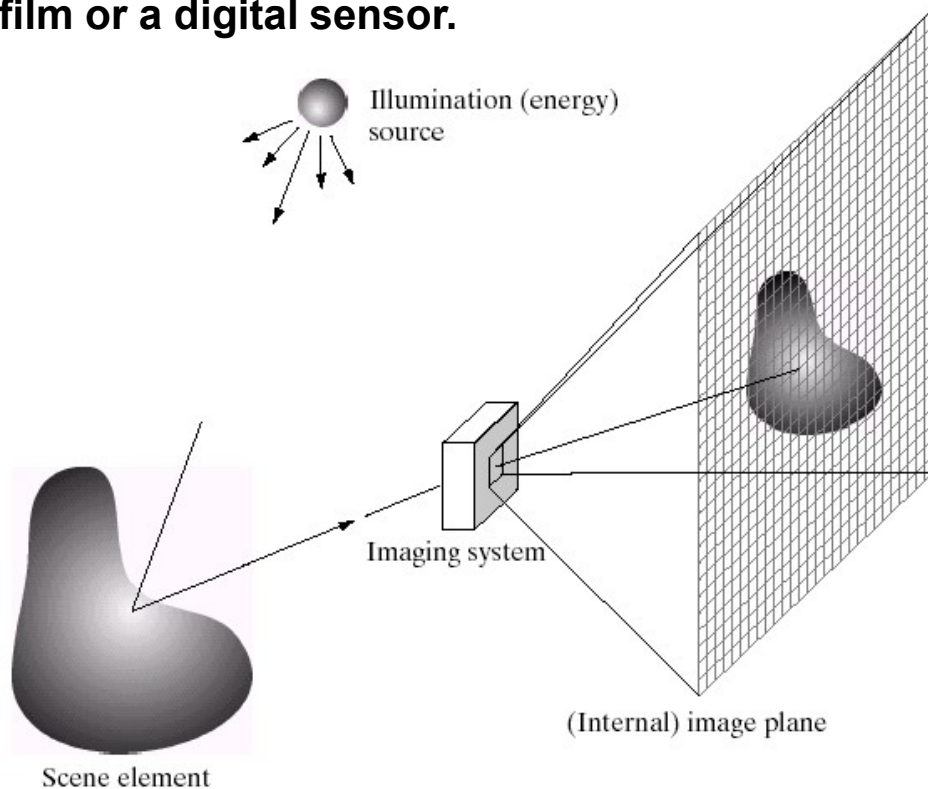
What is an image?



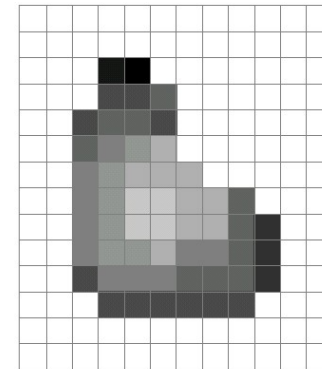
Robert Collins
CSE486, Penn State

What is an image?

A record of a set of light rays that bounce through space and hit a piece of light-sensitive film or a digital sensor.

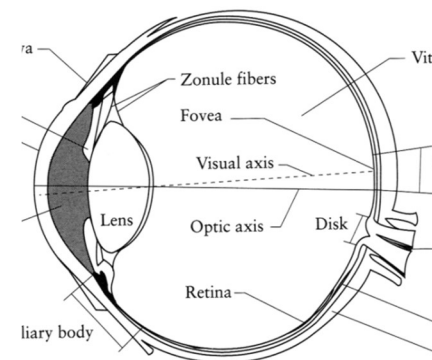


A grid of pixel values.



We focus on this! Digital Images.

A response of our retinas to a light stimulus

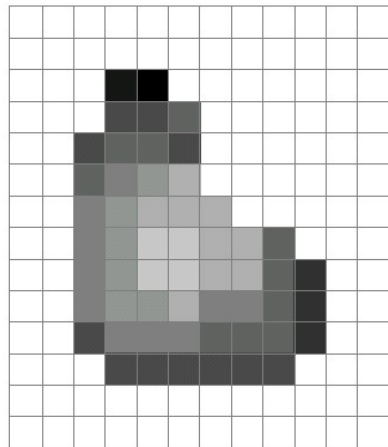


We receive two “images” (one for each eye) and our brain assembles the 3D scene.

Source: A. Efros

What is an image?

- A grid (matrix) of intensity (integer) values



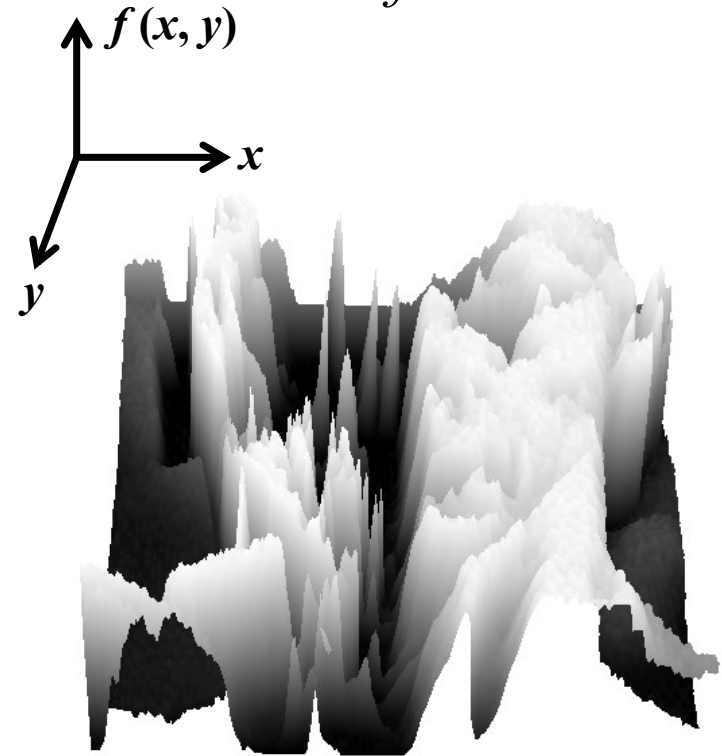
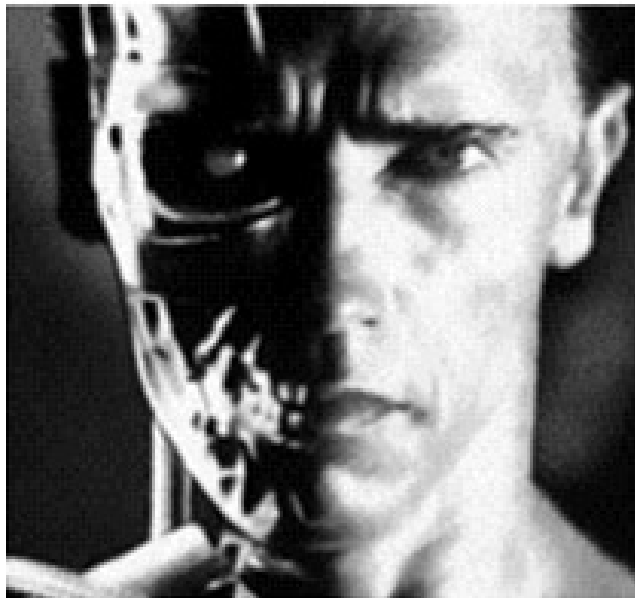
=

255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	20	0	255	255	255	255	255	255	255	255	255	255
255	255	255	75	75	75	255	255	255	255	255	255	255	255	255
255	255	75	95	95	75	255	255	255	255	255	255	255	255	255
255	255	96	127	145	175	255	255	255	255	255	255	255	255	255
255	255	127	145	175	175	175	255	255	255	255	255	255	255	255
255	255	127	145	200	200	175	175	95	255	255	255	255	255	255
255	255	127	145	200	200	175	175	95	47	255	255	255	255	255
255	255	127	145	145	175	127	127	95	47	255	255	255	255	255
255	255	74	127	127	127	95	95	95	47	255	255	255	255	255
255	255	255	74	74	74	74	74	74	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255

(common to use one byte per value: 0 = black, 255 = white)

What is an image?

- Can think of a (grayscale) image as a **function** f from $\mathbb{R}^2$ to $\mathbb{R}$:
 - $f(x,y)$ gives the **intensity** at position (x,y)



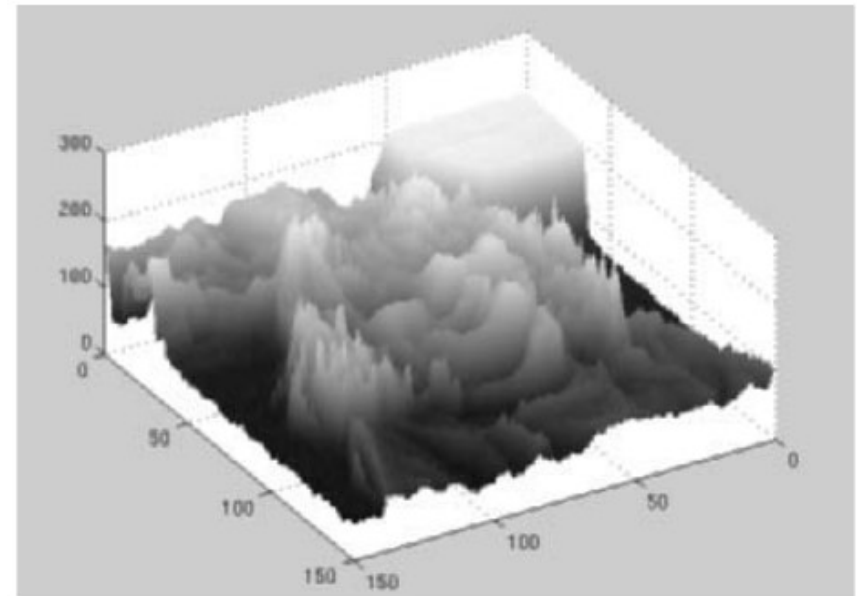
- A **digital** image is a **discrete** (**sampled**, **quantized**) version of this function

We digitize the coordinate values

We digitize the intensities

What is an image?

- Can think of a (grayscale) image as a **function** f from $\mathbb{R}^2$ to $\mathbb{R}$:
 - $f(x,y)$ gives the **intensity** at position (x,y)



Source:
M. Sonka

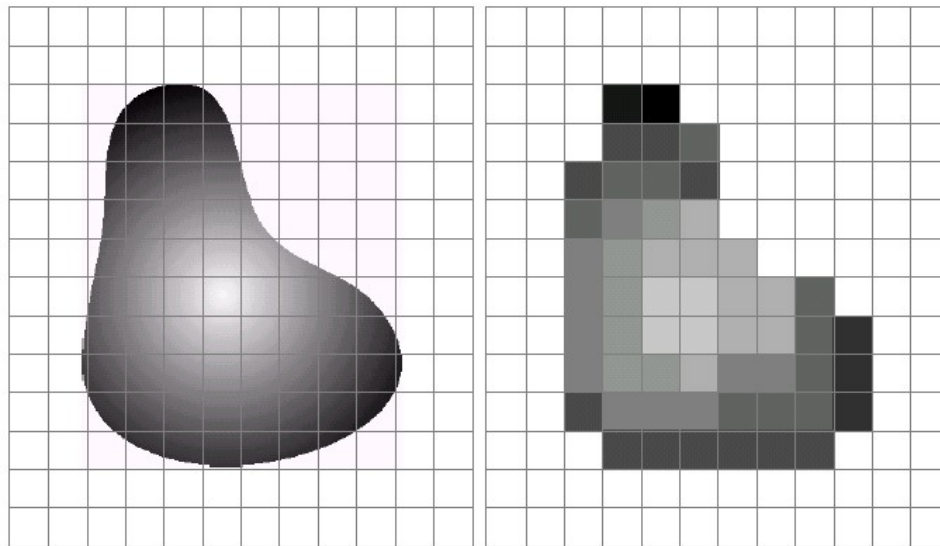
- A **digital** image is a **discrete** (**sampled**, **quantized**) version of this function

We digitize the coordinate values

We digitize the intensities

What is an image?

- A **digital** image is a **discrete (sampled, quantized)** version of this intensity function
 - **Sample** the 2D space on a regular grid
 - **Quantize** each sample (round to nearest integer)



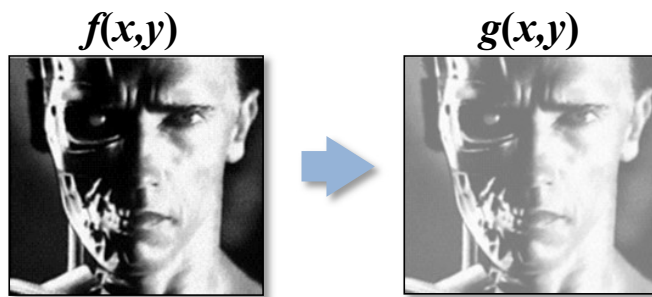
a b

FIGURE 2.17 (a) Continuous image projected onto a sensor array. (b) Result of image sampling and quantization.

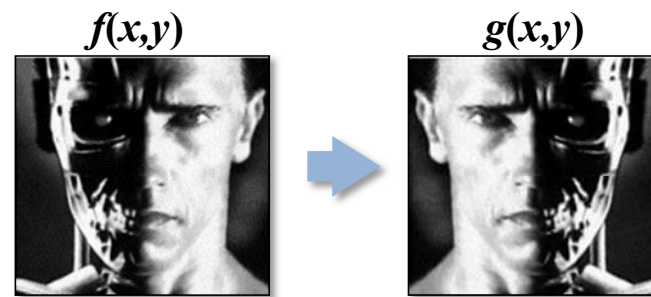
We deal with a
**noisy/approximate
representation of reality**
(inaccurate in terms of
intensity and space)

Image transformations

- As with any function, we can apply (global) operators to an image

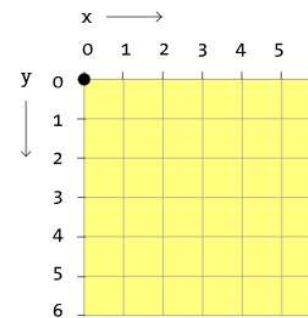


$$g(x,y) = f(x,y) + 20$$



$$g(x,y) = f(-x,y)$$

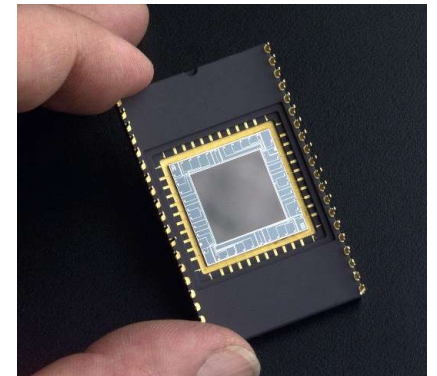
- We'll talk about a special kind of operator: **convolution** (local transformation)



In this example, we assume (0,0) is the center of the image.

Note: generally, (0,0) can be found at the top left with the positive direction to the right horizontally and down vertically.

Digital camera



A digital camera replaces film with an image sensor (CCD or CMOS)

- Each cell in the array is a light-sensitive diode that converts photons to electrons.
- The camera collects light that bounces off objects.
- <http://electronics.howstuffworks.com/digital-camera.htm>

Digital color images

Color images,
RGB color
space



R



G

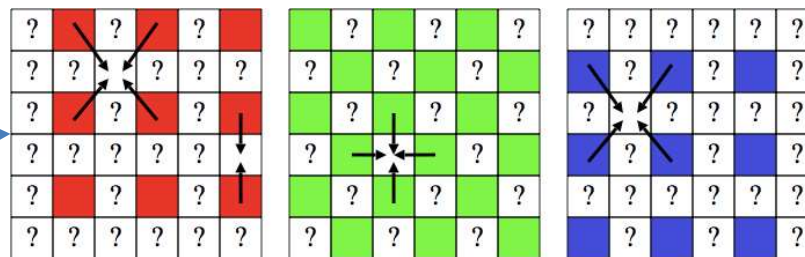
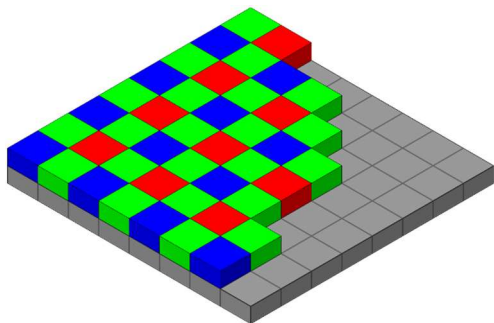


B

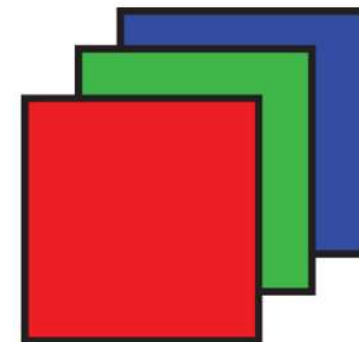
Three gray
level images

Digital color images

Typically, each 2x2 block is encoded as RGGB

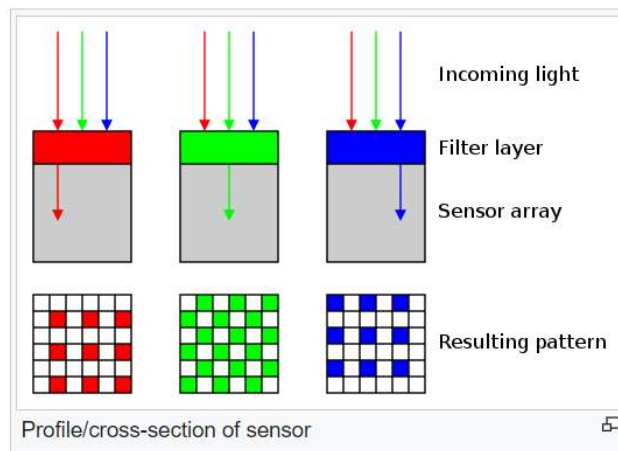


RGB image is computed by interpolation



The **Bayer Filter** is one color filter array (patented in 1976).

Green: 50%, Red: 25%, and Blue: 25%



Mimics the physiology of the human eye (M cones are most sensitive to green light, whereas L cones peak in the yellow-green region).

This strategy is much less expensive than a three-CCD camera (https://en.wikipedia.org/wiki/Three-CCD_camera)

Very interesting video on Color Perception (by Steve Seitz): <https://www.youtube.com/watch?v=cM2wvHtVcM8>

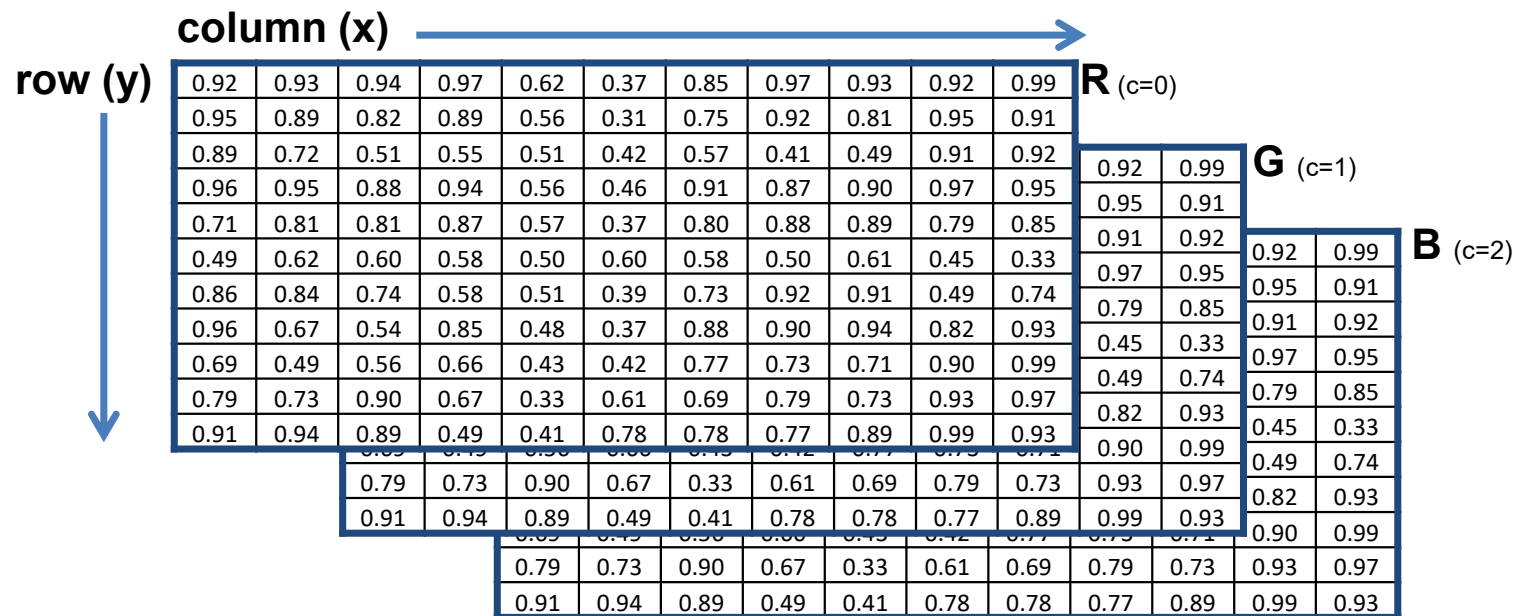
https://en.wikipedia.org/wiki/Bayer_filter

Digital color images in Python

```
im = cv2.imread(filename)           # read image
im = cv2.cvtColor(im, cv2.COLOR_BGR2RGB) # order channels as RGB
im = im / 255                       # values range from 0 to 1
```

- RGB image `im` is a `H x W x 3` matrix (numpy.ndarray)
- `im[0,0,0]` = top-left pixel value in R-channel
- `im[y, x, c]` = pixel at row `y`, column `x`, channel `c`
- `im[H-1, W-1, 2]` = bottom-right pixel in B-channel

Very important to operate with floating-point numbers!



Slide credit:
Derek Hoiem

Filters

- Filtering
 - **Form a new image whose pixel values are a combination of the original pixel values**
- Why?
 - To get useful information from images
 - E.g., extract edges (to understand shape)
 - To enhance images
 - E.g., to remove noise
 - E.g., to sharpen details
 - To detect patterns
 - Template matching
 - A key concept/operator in Convolutional Neural Networks

Image filtering

- We generally modify the pixels in an image based on some function of a **local neighborhood** around each pixel

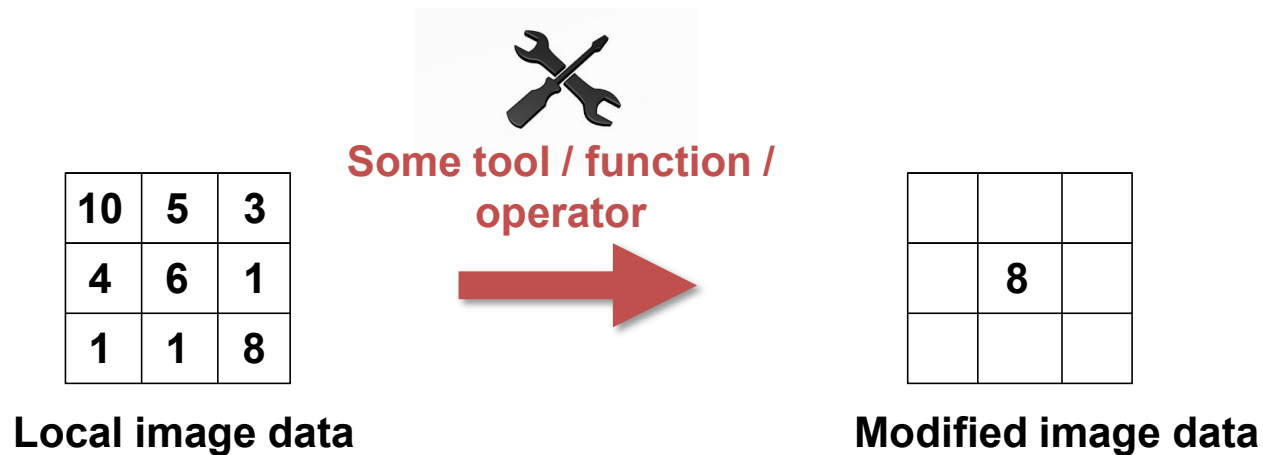


Image filtering

- We generally modify the pixels in an image based on some function of a **local neighborhood** around each pixel
 - Wait... why a local neighborhood?? Why not global or individual???



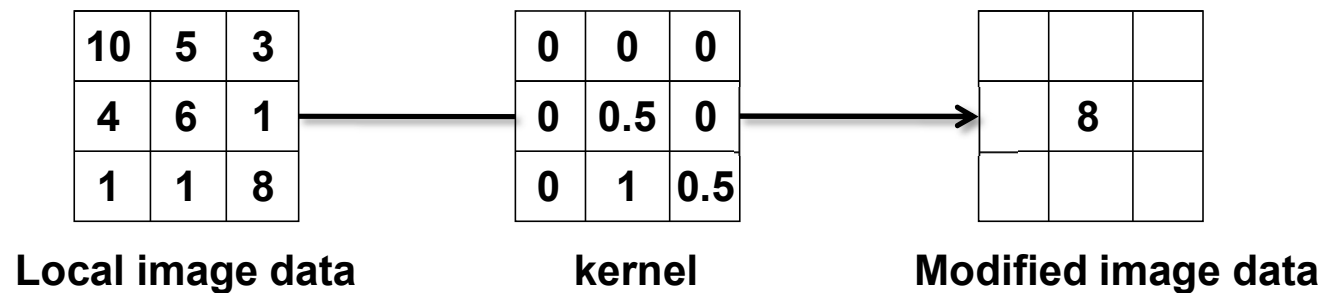
- Nearby pixels show dependence between their values.
- Weaker and less predictable relationships between distant pixels/regions (high spatial variability).
- The isolated and decontextualized analysis of the pixels is worthless. Local information is essential.



Pixels, by themselves, are not relevant. What is relevant is the relationship among pixels.

Linear filtering

- One simple version of filtering: linear filtering
 - Replace each pixel by a linear combination (a weighted sum) of its neighbors (cross-correlation, convolution)
- The weights for the linear combination are called the “kernel” / “mask” / “filter”



Kind reminder:

$$L \text{ is a linear operator iff } L(af + bg) = aL(f) + bL(g)$$

Scalars

Images

Common types of noise

- **Impulse noise:** random occurrences of pixels with values very different from their surroundings.
 - **Salt-and-pepper noise:** random occurrences of both bright and dark pixels.
 - **Salt noise:** random occurrences of bright pixels only.
- **Gaussian noise:** additive intensity variations sampled from a Gaussian (normal) distribution.
- Different noise models require different filtering strategies.



Original



Salt-and-pepper noise



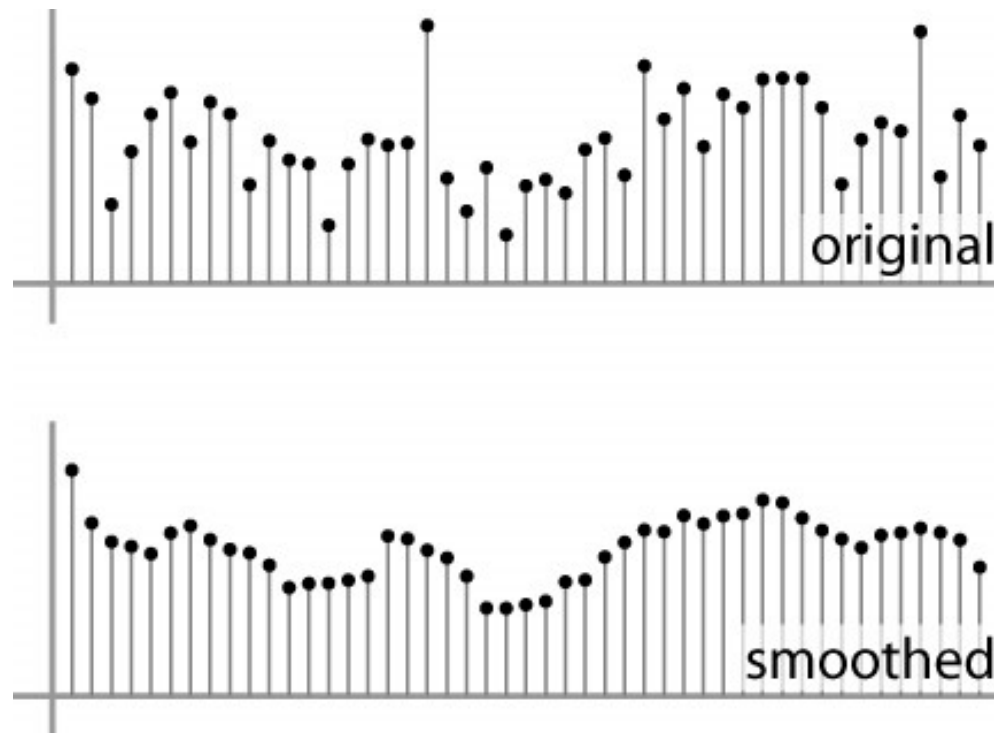
Salt noise



Gaussian noise

Smoothing for noise reduction

- Let's replace each pixel with an average of all the values in its neighborhood
- Moving average in 1D:

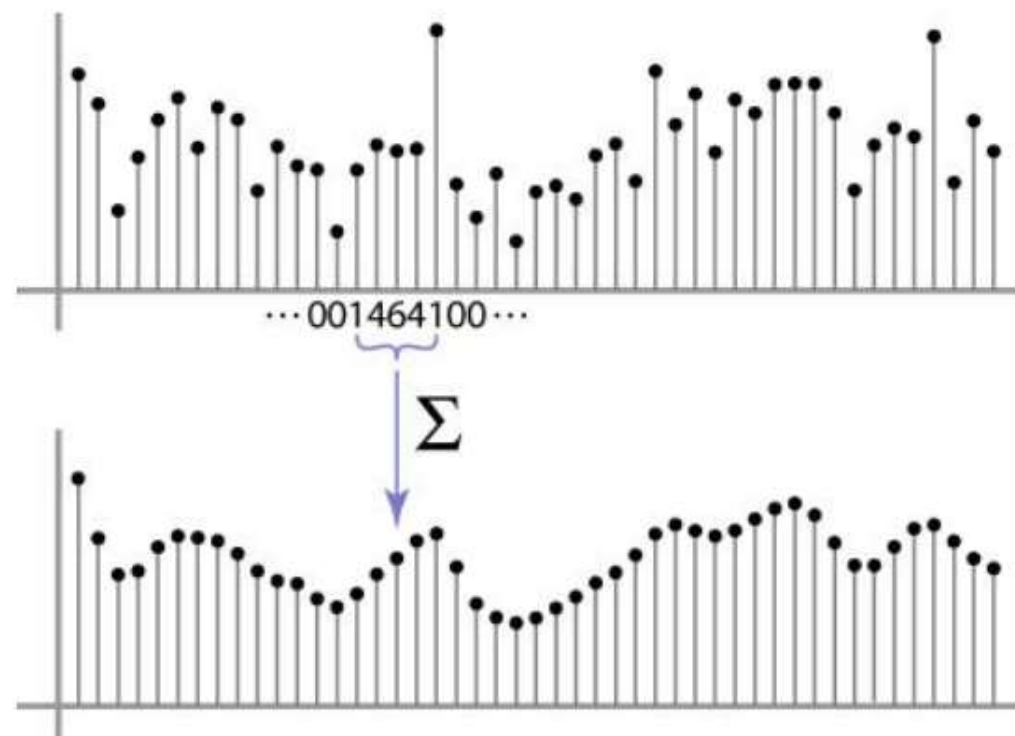
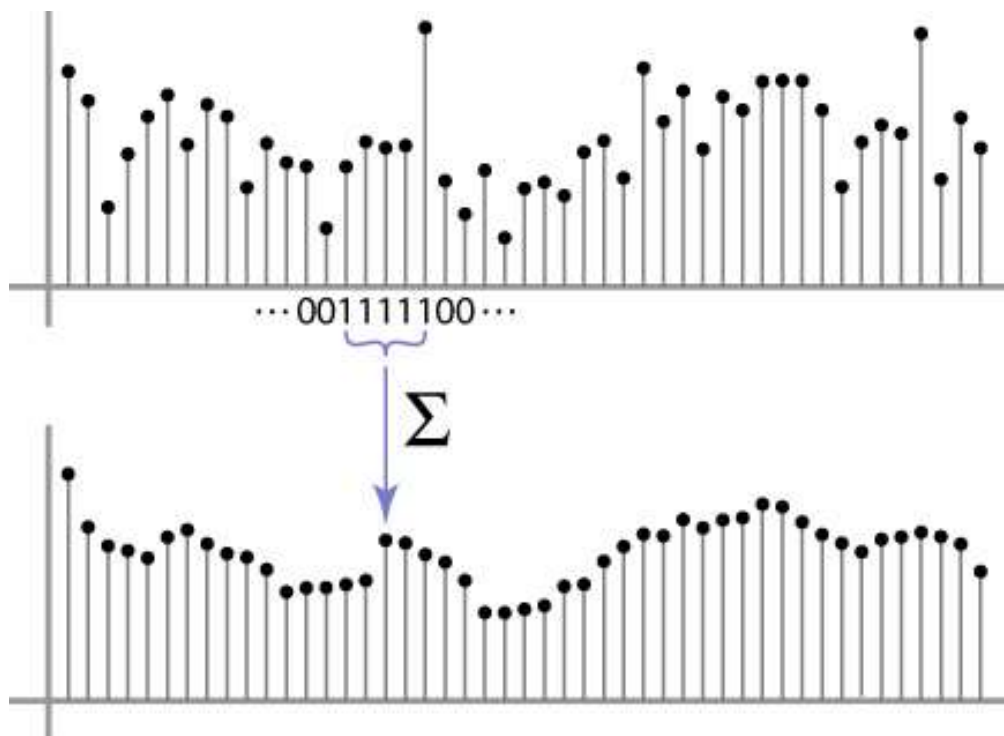


Smoothing for noise reduction

- Can add weights to our moving average. For instance, non-uniform weights

$[1, 4, 6, 4, 1] / 16$

What is the shape of these weights?
Does it look familiar?



Cross-correlation

Are kernel sizes typically even or odd? Why?



Let F be the image, H be the kernel (of size $2k + 1 \times 2k + 1$), and G be the output image

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

This is called a cross-correlation

operation: $G = H \otimes F$

- Can think of as a “dot product” between local neighborhood and kernel for each pixel

Convolution

- Same as cross-correlation, except that the kernel is “flipped” (horizontally & vertically = bottom to top & right to left = rotate it 180°)

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

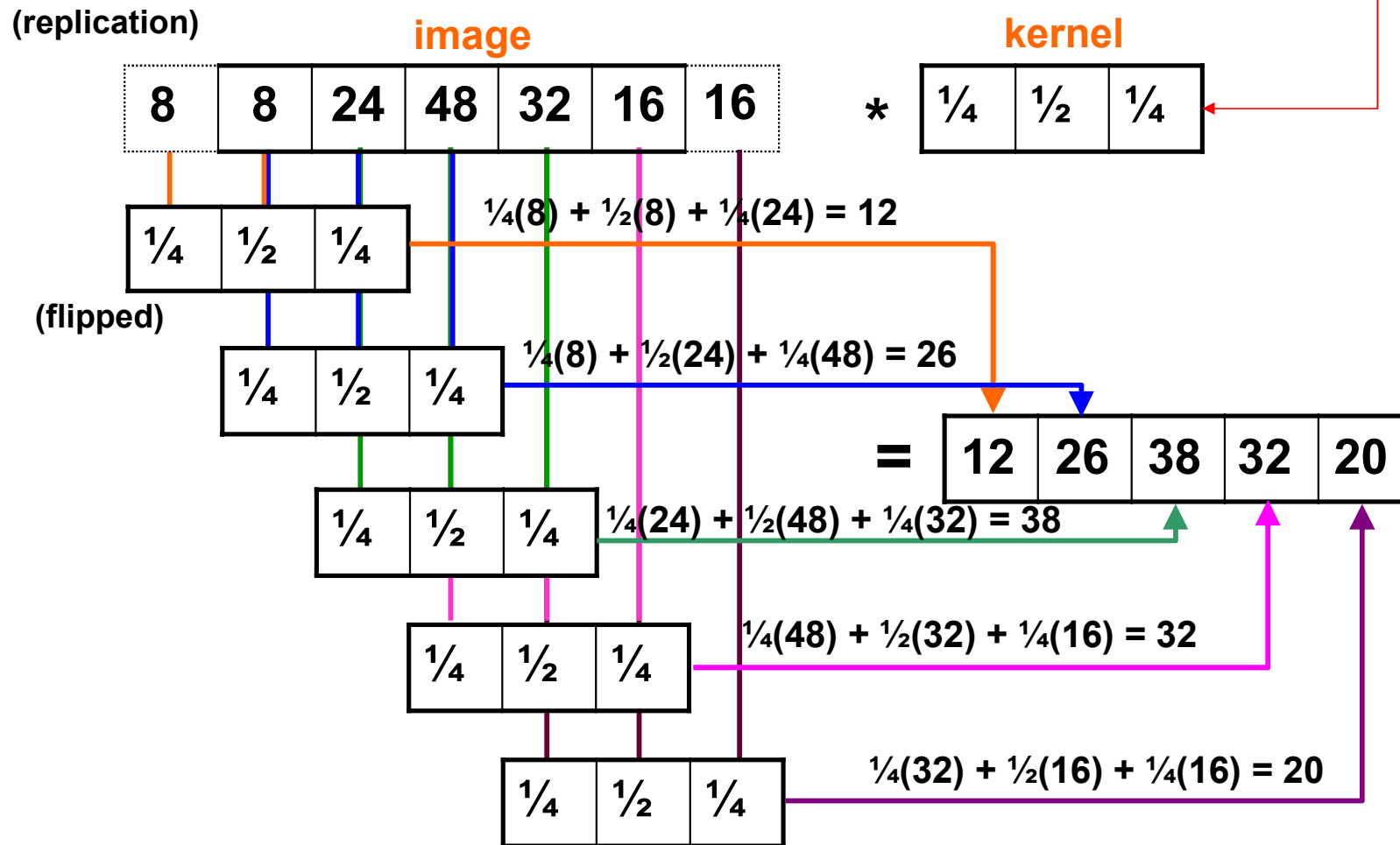
This is called a convolution operation:

$$G = H * F$$

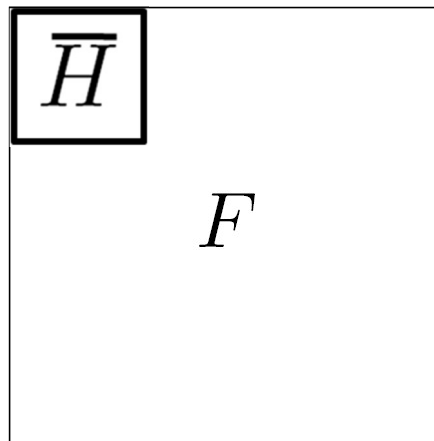
- Convolution is **commutative** and **associative**

1D convolution example

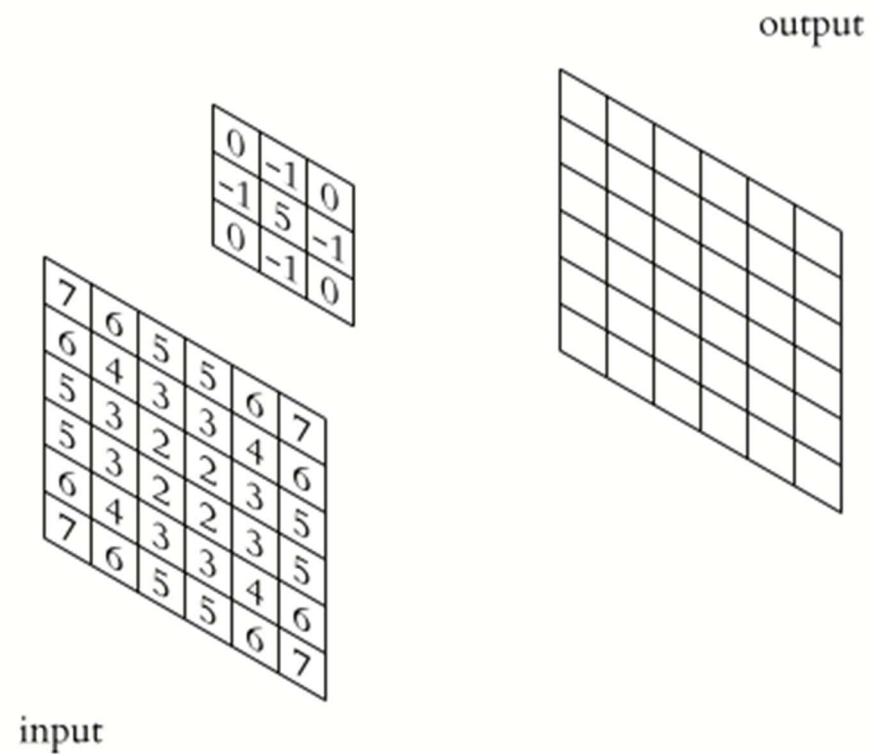
What happens if you flip this kernel? 🤔



2D convolution



2D convolution



https://upload.wikimedia.org/wikipedia/commons/1/19/2D_Convolution_Animation.gif

(using replicated boundaries)

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 1 & 2 & 3 & \\ \hline 4 & 50 & 60 & 0 \\ \hline 7 & 80 & 91 & 0 \\ \hline & 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 40 & 50 & 60 \\ \hline 70 & 81 & 90 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline & 1 & 2 & 3 \\ \hline 0 & 4 & 0 & 5 & 0 & 6 \\ \hline 0 & 7 & 1 & 8 & 0 & 9 \\ \hline 0 & 0 & 0 & & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & 7 \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

1	20	30	0
4	50	61	0
7	80	90	0

=

9	8	7
6		

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

10	20	30
40	51	60
70	80	90

=

9	8	7
6	5	

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

0	10	20	3
0	41	50	6
0	70	80	9

=

9	8	7
6	5	4

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

		0	0	0
1	2	0	3	1
4	5	0	6	0
7	8	9		

=

9	8	7
6	5	4
3		

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

0	0	0
1 0	2 1	3 0
4 0	5 0	6 0
7	8	9

=

9	8	7
6	5	4
3	2	

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

0	0	0	
0	1 1	2 0	3
0	4 0	5 0	6
	7	8	9

=

9	8	7
6	5	4
3	2	1

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 11 & 02 \\ \hline 0 & 04 & 05 \\ \hline & 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H & & \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 1 & 1 2 & 0 3 \\ \hline 0 4 & 0 5 & 0 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline & 0 & 0 & 0 \\ \hline 1 & 0 & 2 & 1 & 3 & 0 \\ \hline 4 & 0 & 5 & 0 & 6 & 0 \\ \hline 7 & 8 & 9 & & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

0	01	02	3
0	14	05	6
0	07	08	9

=

1	2	3
4		

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

0 1	0 2	0 3
0 4	1 5	0 6
0 7	0 8	0 9

=

1	2	3
4	5	

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

1	0	2	0	3	0
4	0	5	1	6	0
7	0	8	0	9	0

=

1	2	3
4	5	6

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

		1	2	3
0	0	4	5	6
0	1	7	8	9
0	0	0		

=

1	2	3
4	5	6
7		

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

1	2	3
0 4	0 5	0 6
0 7	1 8	0 9
0	0	0

=

1	2	3
4	5	6
7	8	

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

1	2	3	
4	0	5	0
7	0	8	1
	0	0	0

=

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & 7 \\ \hline 6 & 5 & 4 \\ \hline 3 & 2 & 1 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

Not commutative!!!!

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 9 & 8 & 7 & \\ \hline 6 & 50 & 40 & 0 \\ \hline 3 & 20 & 11 & 0 \\ \hline & 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & 7 \\ \hline 60 & 50 & 40 \\ \hline 30 & 21 & 10 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline & 9 & 8 & 7 \\ \hline 0 & 60 & 50 & 4 \\ \hline 0 & 31 & 20 & 1 \\ \hline 0 & 0 & 0 & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

9	80	70	0
6	50	41	0
3	20	10	0

=

1	2	3
4		

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

9 0	8 0	7 0
6 0	5 1	4 0
3 0	2 0	1 0

=

1	2	3
4	5	

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

0	9	0	8	0	7
0	6	1	5	0	4
0	3	0	2	0	1

=

1	2	3
4	5	6

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

		0	0	0
9	8	0	7	1
6	5	0	4	0
3	2	1		

=

1	2	3
4	5	6
7		

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

0	0	0
9 0	8 1	7 0
6 0	5 0	4 0
3	2	1

=

1	2	3
4	5	6
7	8	

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

0	0	0	
0	91	80	7
0	60	50	4
	3	2	1

=

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 11 & 02 \\ \hline 0 & 04 & 05 \\ \hline & 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H & & \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F & & \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 1 & 2 & 0 & 3 \\ \hline 0 & 4 & 0 & 5 & 0 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H & & \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F & & \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline & 0 & 0 & 0 \\ \hline 1 & 0 & 2 & 1 & 3 & 0 \\ \hline 4 & 0 & 5 & 0 & 6 & 0 \\ \hline 7 & 8 & 9 & & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

0	01	02	3
0	14	05	6
0	07	08	9

=

1	2	3
4		

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

0 1	0 2	0 3
0 4	1 5	0 6
0 7	0 8	0 9

=

1	2	3
4	5	

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	0	2	0	3	0
4	0	5	1	6	0
7	0	8	0	9	0

=

1	2	3
4	5	6

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

		1	2	3	
0	0	4	0	5	6
0	1	7	0	8	9
0	0	0			

=

1	2	3
4	5	6
7		

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	2	3
0 4	0 5	0 6
0 7	1 8	0 9
0	0	0

=

1	2	3
4	5	6
7	8	

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	2	3	
4	0	5	0
7	0	8	1
	0	0	0

=

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

Commutative!!!!

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \text{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \text{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & 7 \\ \hline 6 & 5 & 4 \\ \hline 3 & 2 & 1 \\ \hline \end{array}$$

$5*0 + 6*0 + 8*0 + 9*1$ $4*0 + 5*0 + 6*0 + 7*0 + 8*1 + 9*0$

If the input is an impulse signal, cross-correlation produces the filter rotated 180°. **Convolution gives a more natural output:** if the mask is convolved with an impulse signal, it provides exactly the same mask as output.

Convolution

$$\begin{array}{|c|c|c|} \hline \text{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \text{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline \text{Kernel } H \\ \text{(rotated } 180^\circ) \\ \hline 9 & 8 & 7 \\ \hline 6 & 5 & 4 \\ \hline 3 & 2 & 1 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \text{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

$5*0 + 4*0 + 2*0 + 1*1$ $6*0 + 5*0 + 4*0 + 3*0 + 2*1 + 1*0$

Cross-correlation vs Convolution (technical note)

- Functions like `cv2.filter2D` actually apply cross-correlation and not convolution!!!

- Other functions, like `scipy.signal.convolve2d`, do flip the kernel.

`cv2.filter2D(im, -1, cv2.flip(fil, -1))`

equivalent to

`signal.convolve2d(im, fil, mode='same')`

To obtain exactly the same result with both functions, we also need to use compatible boundary conditions.

- So, please, check the documentation before using the functions!!!

Cross-correlation vs Convolution (technical note)

```
import scipy.signal as sp
import numpy as np
import cv2

image = np.array([[0, 0, 0], [0,1,0],[0,0,0]], dtype=np.uint8)
filter = np.array([[1, 2, 3], [4,5,6],[7,8,9]], dtype=np.uint8)

print('Result using sp.convolve2d')
print(sp.convolve2d(image, filter, mode='same'))

print('Result using cv2.filter2D')
print(cv2.filter2D(image, -1, filter, borderType=cv2.BORDER_CONSTANT))

print('Result using cv2.filter2D and cv2.flip')
print(cv2.filter2D(image, -1, cv2.flip(filter,-1), borderType=cv2.BORDER_CONSTANT))
```

```
Result using sp.convolve2d
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Result using cv2.filter2D
[[9 8 7]
 [6 5 4]
 [3 2 1]]
Result using cv2.filter2D and cv2.flip
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Cross-correlation vs Convolution (technical note)

```
import scipy.signal as sp  
import cv2
```

```
sp.convolve2d()
```

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.convolve2d.html>

```
sp.correlate2d()
```

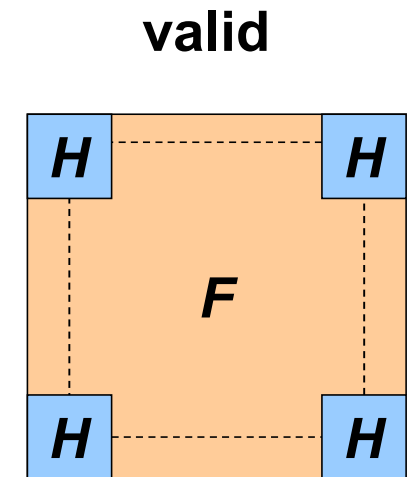
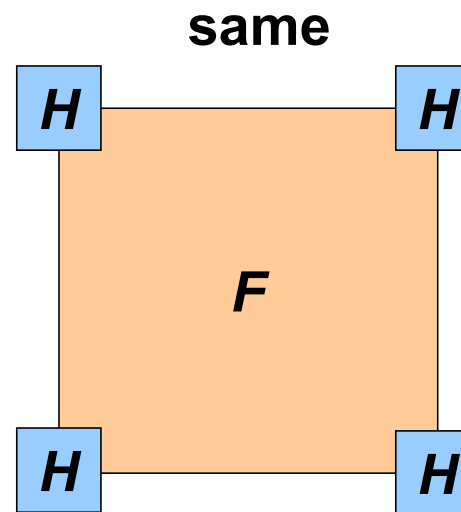
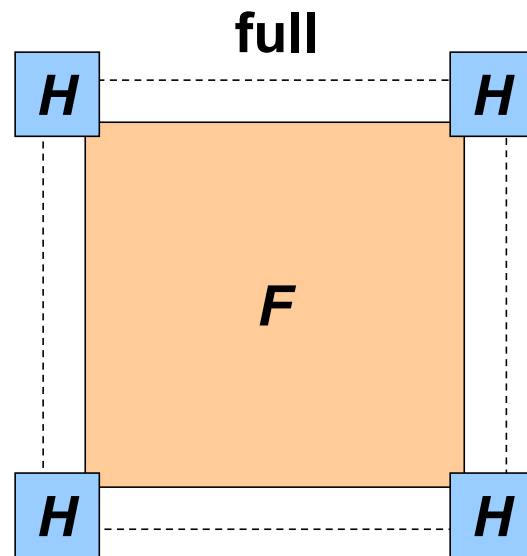
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.correlate2d.html>

```
cv2.filter2D()
```

https://docs.opencv.org/4.x/d4/d86/group_imgproc_filter.html#ga27c049795ce870216ddfb366086b5a04

Boundary issues

- What is the size of the output?
- Output size / “shape” options



Kernel H

1	2	3
4	5	6
7	8	9

Image F



0	0	0
0	1	0
0	0	0

0	0	0	0	0
0	1	2	3	0
0	4	5	6	0
0	7	8	9	0
0	0	0	0	0

1	2	3
4	5	6
7	8	9

5

Boundary issues

- What about near the edge?
 - the filter window falls off the edge of the image
 - need to extrapolate
 - methods:
 - clip filter (black) (a.k.a. zero padding)
 - wrap around
 - copy edge
 - reflect across edge



Boundary issues

- What about near the edge?
 - the filter window falls off the edge of the image
 - need to extrapolate
 - Types (OpenCV):

* *BORDER_REPLICATE*: *aaaaaa | abcdefgh | hhhhhhh*

* *BORDER_REFLECT*: *fedcba | abcdefgh | hgfedcb*

* *BORDER_REFLECT_101*: *gfedcb | abcdefgh | gfedcba* ← Default value

* *BORDER_WRAP*: *cdefgh | abcdefgh | abcdefg*

* *BORDER_CONSTANT*: *iiiiii | abcdefgh | iiiiii*

Cross-correlation vs Convolution

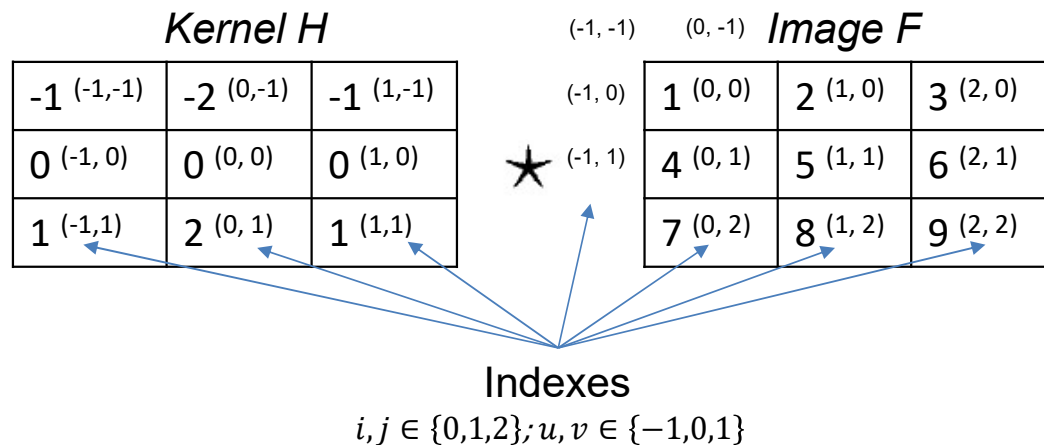
Cross-correlation

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

Convolution

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

How exactly do these indices work? Let's see an example using cross-correlation (convolution is the same: just flip the filter in both dimensions, then apply cross-correlation)



$$\begin{aligned}
 G[0,0] &= H[-1, -1] \cdot F[-1, -1] + H[-1, 0] \cdot F[-1, 0] + H[-1, 1] \cdot F[-1, 1] \\
 &\quad + H[0, -1] \cdot F[0, -1] + H[0, 0] \cdot F[0, 0] + H[0, 1] \cdot F[0, 1] + H[1, -1] \\
 &\quad \cdot F[1, -1] + H[1, 0] \cdot F[1, 0] + H[1, 1] \cdot F[1, 1] \\
 &= -1 \cdot 0 + 0 \cdot 0 + 1 \cdot 0 + (-2) \cdot 0 + 0 \cdot 1 + 2 \cdot 4 + (-1) \cdot 0 + 0 \cdot 2 + 1 \cdot 5 \\
 &= 13
 \end{aligned}$$

$$\begin{aligned}
 G[1,1] &= H[-1, -1] \cdot F[0, 0] + H[-1, 0] \cdot F[0, 1] + H[-1, 1] \cdot F[0, 2] + H[0, -1] \\
 &\quad \cdot F[1, 0] + H[0, 0] \cdot F[1, 1] + H[0, 1] \cdot F[1, 2] + H[1, -1] \cdot F[2, 0] + H[1, 0] \\
 &\quad \cdot F[2, 1] + H[1, 1] \cdot F[2, 2] \\
 &= -1 \cdot 1 + 0 \cdot 4 + 1 \cdot 7 + (-2) \cdot 2 + 0 \cdot 5 + 2 \cdot 8 + (-1) \cdot 3 + 0 \cdot 6 + 1 \cdot 9 \\
 &= 24
 \end{aligned}$$

Note: The important thing is that you perform the dot product of an image and a filter (in the case of convolution, rotated 180°), and replace the central value.

Nice reference on convolution:

<https://www.songho.ca/dsp/convolution/convolution.html>

Key properties of convolution

Linearity: if f and g are images, a and b scalars, and L a linear filter/operator:

$$L(af+bg) = aL(f) + bL(g) \quad \left\{ \begin{array}{l} \text{homogeneity (or scaling): } L(af) = aL(f) \\ + \\ \text{additivity: } L(f+g) = L(f) + L(g) \end{array} \right.$$

Shift equivariance: same behavior regardless of pixel location. Operator behaves the same everywhere, i.e. the value of the output depends on the pattern in the image neighborhood, not the position of the neighborhood.

$$L(\text{shift}(f)) = \text{shift}(L(f))$$

In the case of shift invariance we'd have: $L(\text{shift}(f)) = L(f)$

More properties of convolution

- Commutative: $a * b = b * a$
 - Conceptually no difference between filter and signal (image)
- Associative: $a * (b * c) = (a * b) * c$
 - Often apply several filters one after another: $((a * b_1) * b_2) * b_3$
 - This is equivalent to applying one filter: $a * (b_1 * b_2 * b_3)$
- Distributes over addition: $a * (b + c) = (a * b) + (a * c)$
- Scalars factor out: $ka * b = a * kb = k(a * b)$
- Identity: unit impulse $e = [0, 0, 1, 0, 0]$,
 $a * e = a$

Cross-correlation vs Convolution

Cross-correlation	Convolution
Measurement of similarity between two signals/sequences. Commonly used for template matching.	Linear filtering: computes the effect of a filter on a signal.
If the filter is symmetric under a 180° rotation , like a Gaussian or a Laplacian, it makes no difference . When the filter is not symmetric, like a derivative, it makes a difference.	
Not associative.	Associative
Not commutative.	Commutative.
Useful when the goal is to compare an image with a template or pattern . No kernel flipping is required.	Associativity means that a sequence of convolutional filters can be represented by a single equivalent filter: $(f * g) * h = f * (g * h)$ Associativity is especially useful for understanding and designing combinations of filters and for choosing alternative implementations. Computational savings arise only in particular cases, for example when additional structure such as separability can be exploited.

Linear filters: examples



Image



0	0	0
0	1	0
0	0	0

= ?

mode='same' con zero padding

Linear filters: examples



Image



0	0	0
0	1	0
0	0	0



Identical image

mode='same' con zero padding

Linear filters: examples



Image



0	0	0
0	0	1
0	0	0

= ?

mode='same' con zero padding

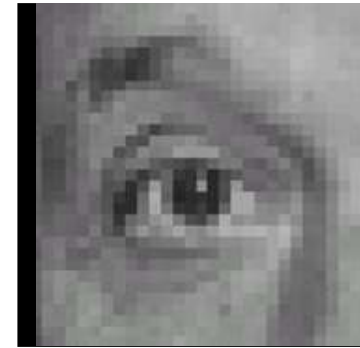
Linear filters: examples



Image



0	0	0
0	0	1
0	0	0



Shifted right by 1 pixel
(we lose the first column of pixels on the right)

mode='same' con zero padding

What if this was a correlation operation???



Image

$$\otimes \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 0 & 1 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = ?$$

mode='same' con zero padding

What if this was a correlation operation???



Image



0	0	0
0	0	1
0	0	0



Shifted left by 1 pixel
(we lose the first column of pixels on the left)

mode='same' con zero padding

Linear filters: examples

Image

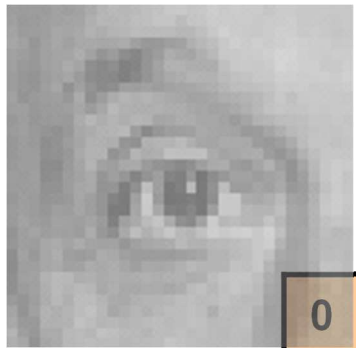
0	0	0
0	0	1
0	0	0

Kernel

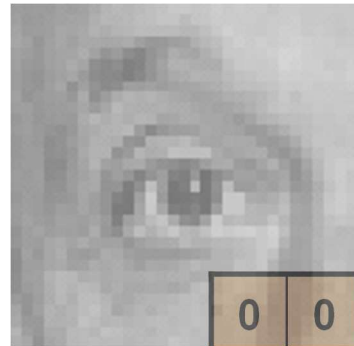


$\otimes$ = ?

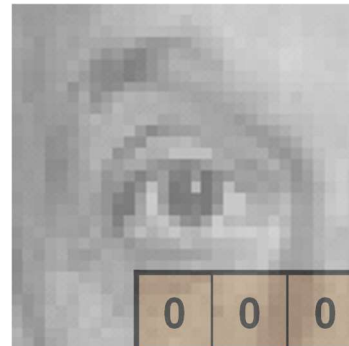
mode='full' con zero padding



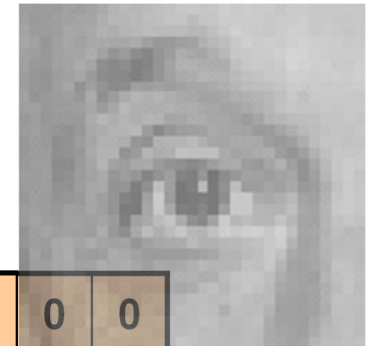
0	0	0
0	0	1
0	0	0



0	0	0
0	0	1
0	0	0



0	0	0
0	0	1
0	0	0

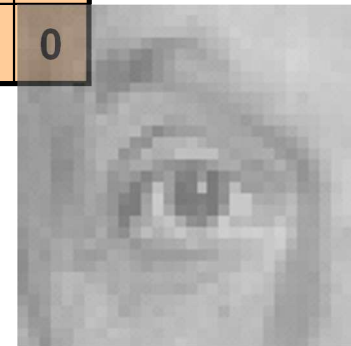


0	0	0
0	0	1
0	0	0

0	0	0
0	0	1
0	0	0



0	0	0
0	0	1
0	0	0

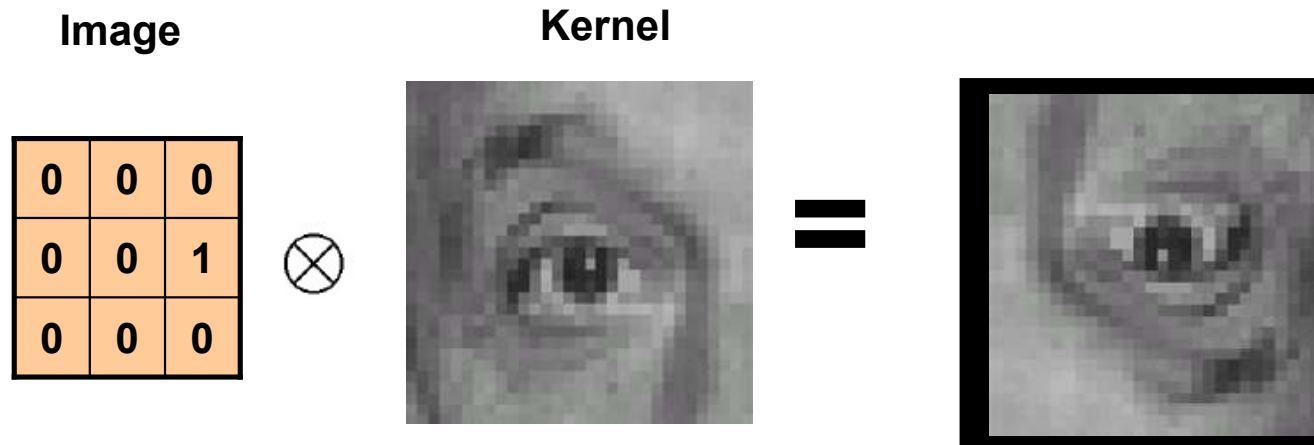


...

This visualization illustrates the main intuition of the operation. It should be noted that the diagram is not to scale; in practice, each position of the kernel (represented by Einstein's eye) would correspond to a single image pixel.

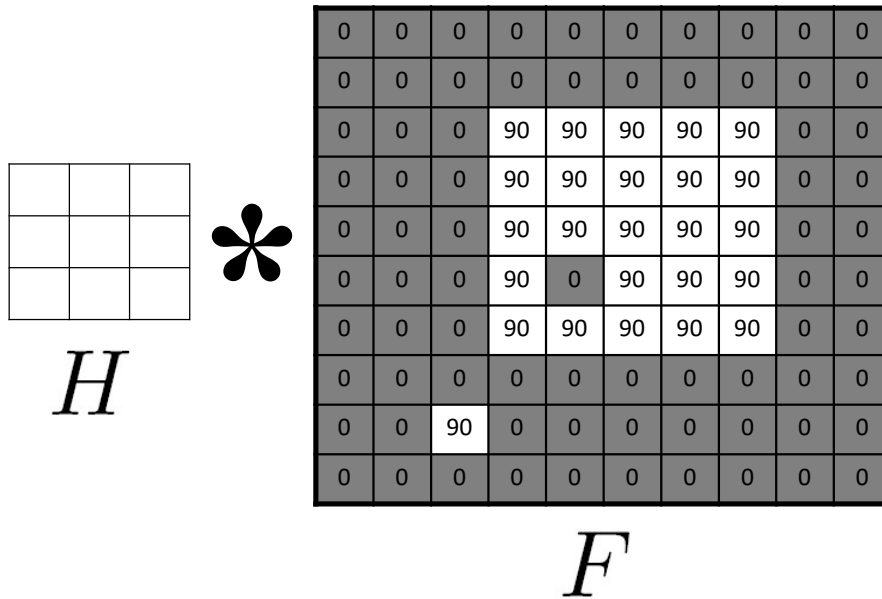
mode='full' con zero padding

Linear filters: examples

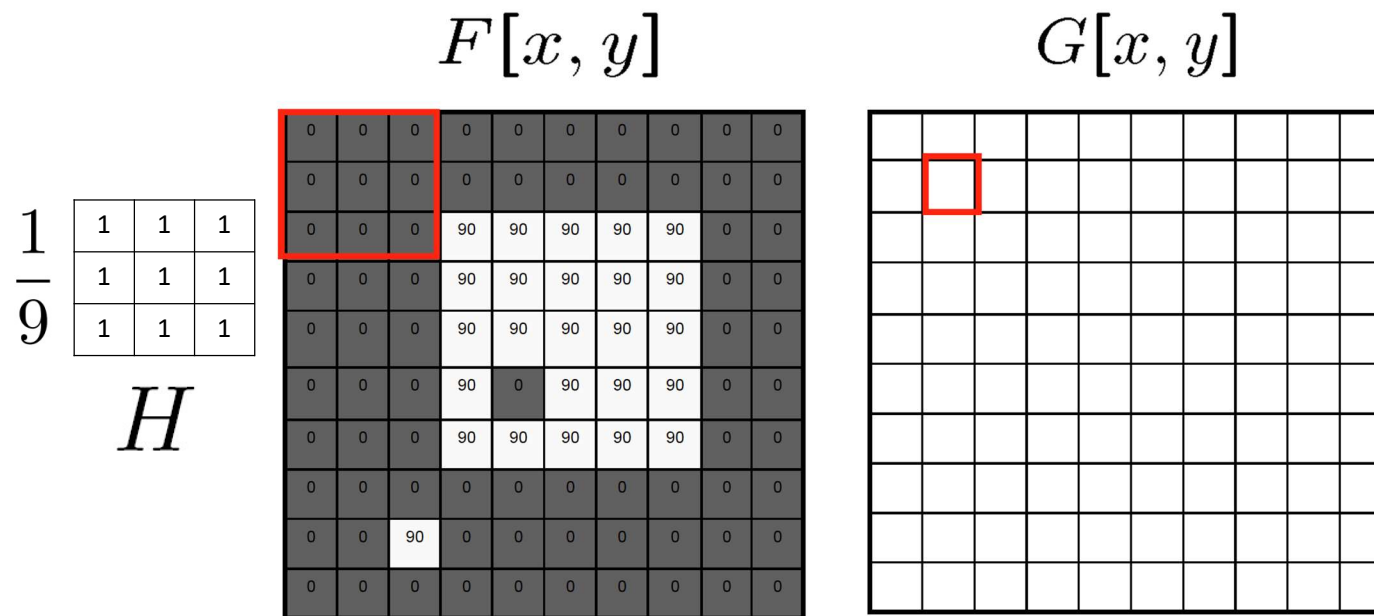


mode='full' con zero padding

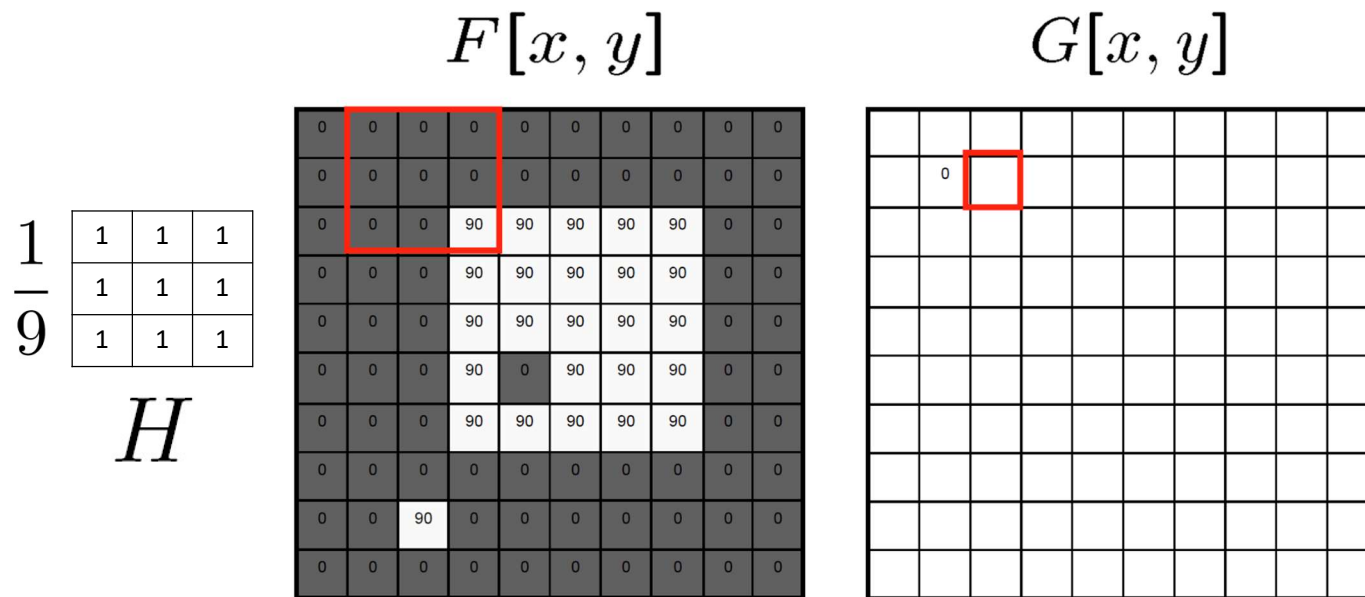
Mean filtering



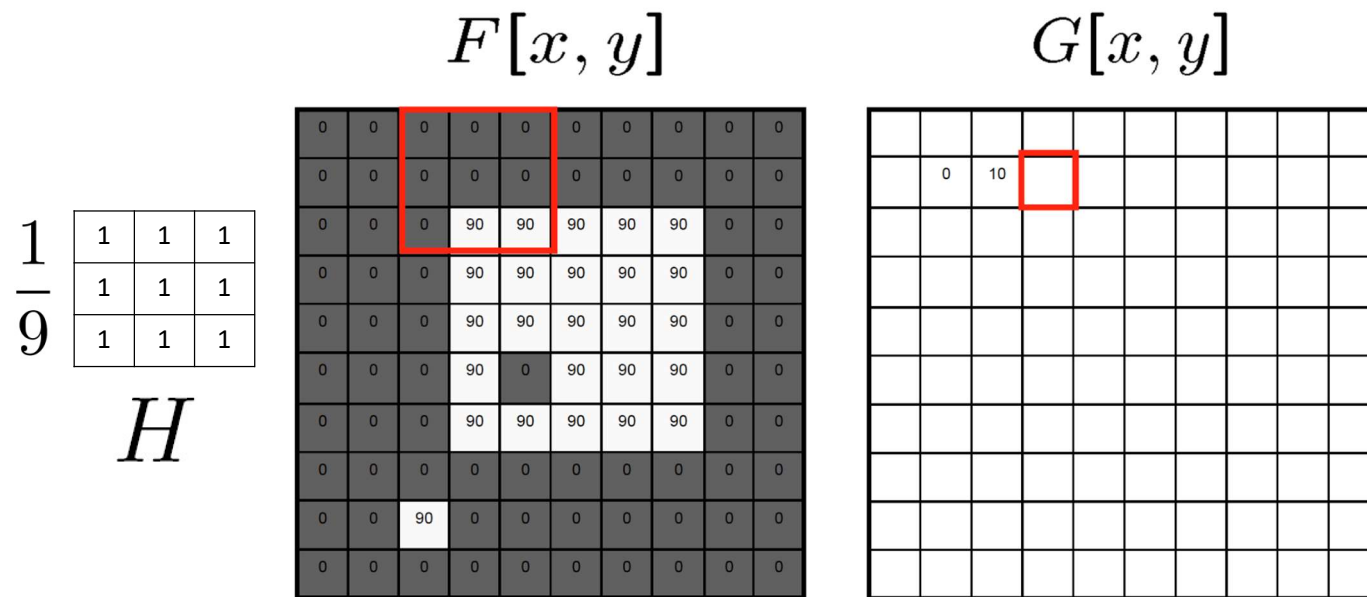
Mean filtering/Moving average



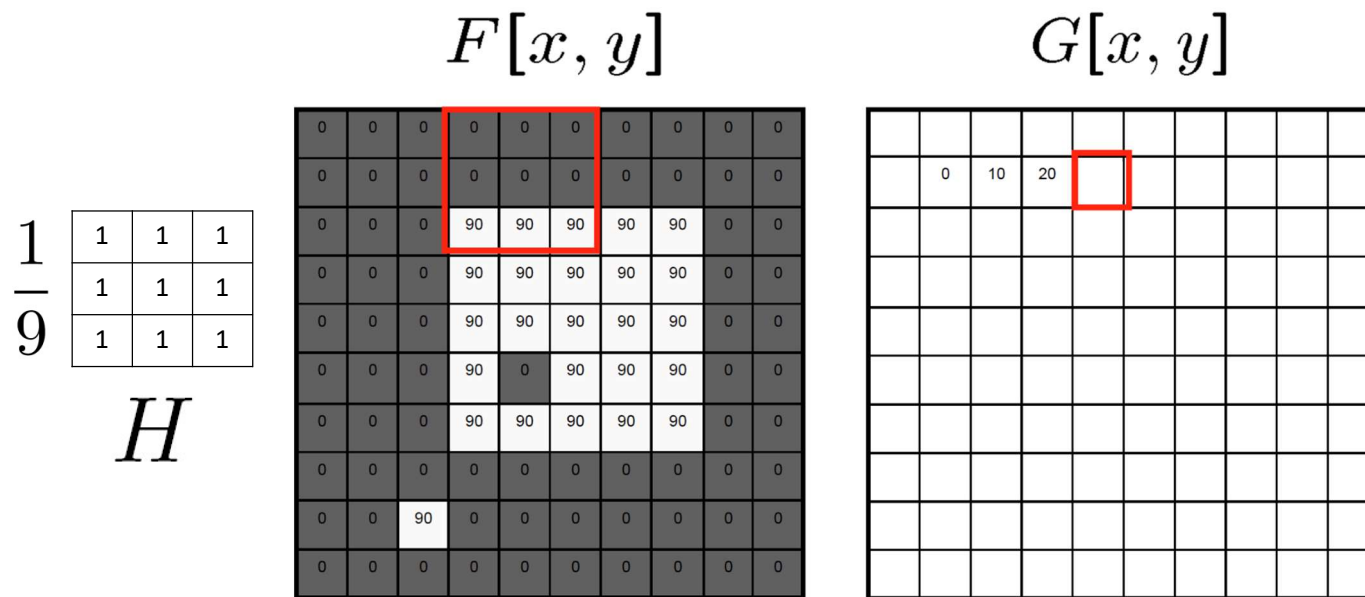
Mean filtering/Moving average



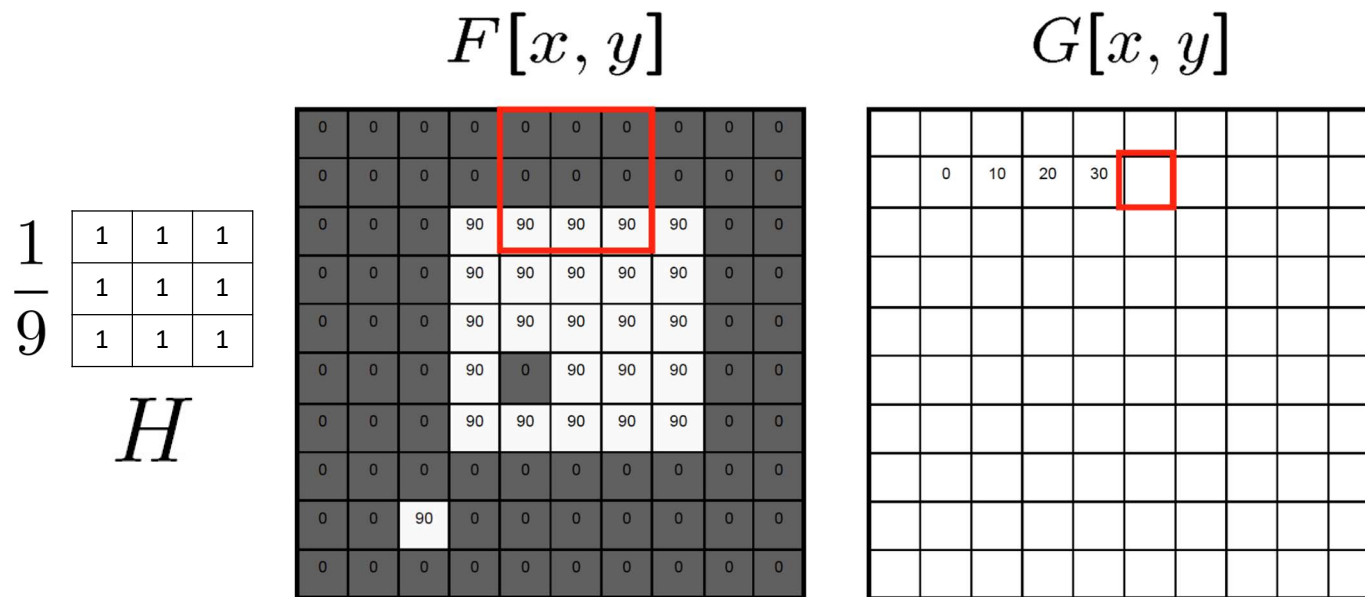
Mean filtering/Moving average



Mean filtering/Moving average



Mean filtering/Moving average



Mean filtering

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} * \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 90 & 90 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 90 & 90 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 90 & 90 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 90 & 0 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 90 & 90 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 90 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} & & & & & & & & & \\ & 0 & 10 & 20 & 30 & 30 & 30 & 20 & 10 & \\ & 0 & 20 & 40 & 60 & 60 & 60 & 40 & 20 & \\ & 0 & 30 & 60 & 90 & 90 & 90 & 60 & 30 & \\ & 0 & 30 & 50 & 80 & 80 & 90 & 60 & 30 & \\ & 0 & 30 & 50 & 80 & 80 & 90 & 60 & 30 & \\ & 0 & 20 & 30 & 50 & 50 & 60 & 40 & 20 & \\ 10 & 20 & 30 & 30 & 30 & 30 & 20 & 10 & & \\ 10 & 10 & 10 & 0 & 0 & 0 & 0 & 0 & & \\ & & & & & & & & & \end{bmatrix}$$

H F G

$\frac{1 \cdot 90}{9} + \frac{1 \cdot 90}{9} = 10 + 10 = 20$

Mean filtering/Moving average

What happens if we don't divide by 9?? 🤔

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1

H

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$G[x, y]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

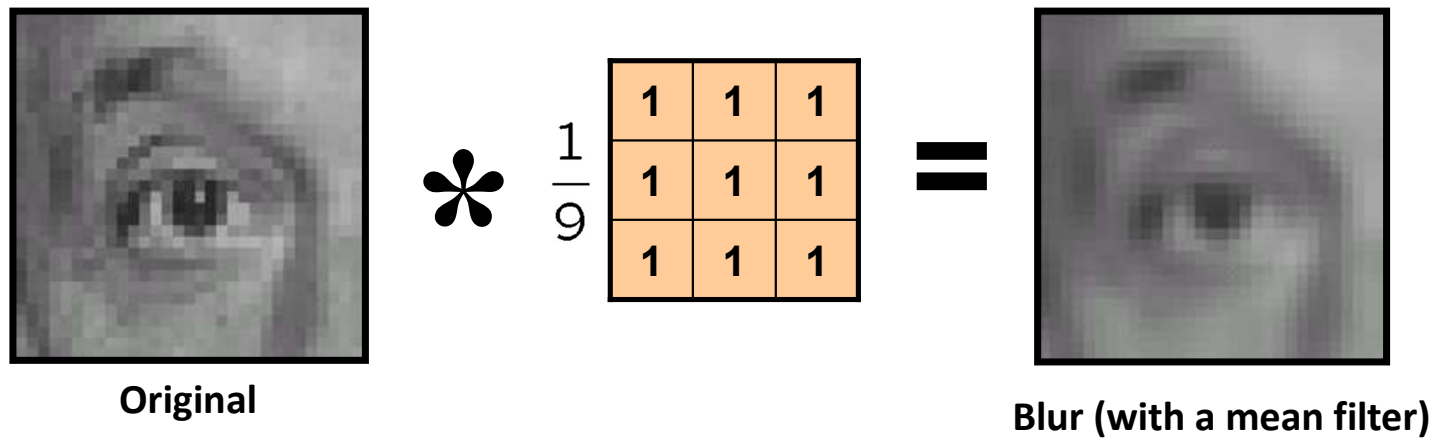
Why is the output image smaller than the input image? 🤔

Would there be any difference if we applied correlation instead of convolution? 🤔

What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieve smoothing effect (remove sharp features)

Linear filters: examples



“box filter” (all mask values are equal): linear filter in which each pixel in the resulting image has a value equal to the average value of its neighboring pixels in the input image

Linear filters: examples



Original

$$* \left(\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \right) =$$

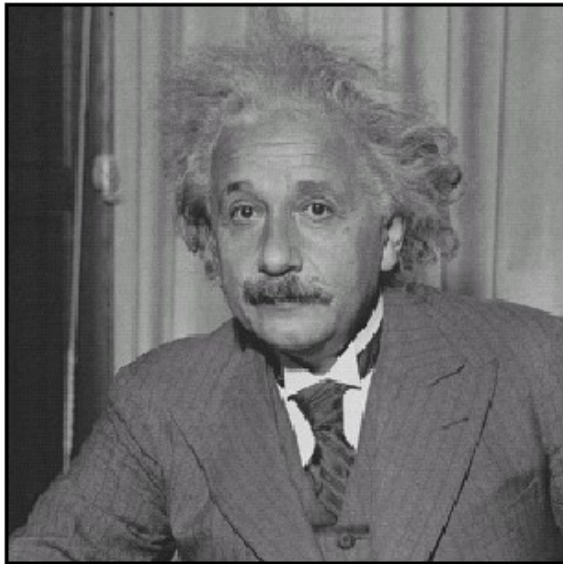
We scale the intensities of the original image

We smooth the image (i.e. we remove high frequencies → we keep low frequencies)

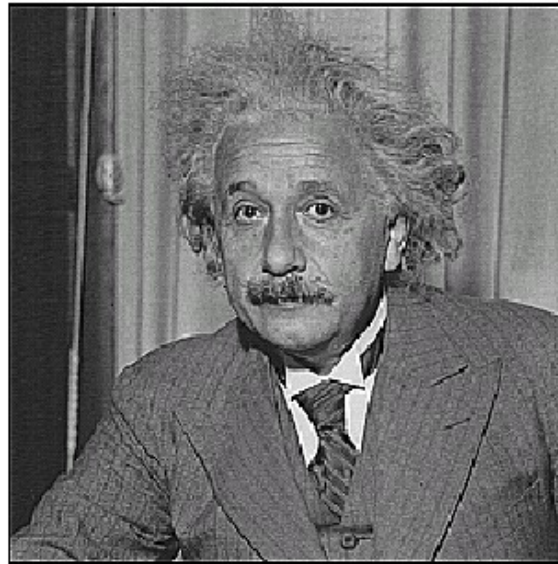


Sharpening filter
(accentuates edges)

Sharpening



before



after

Source image



Sharpened Image



Highly sharpened image



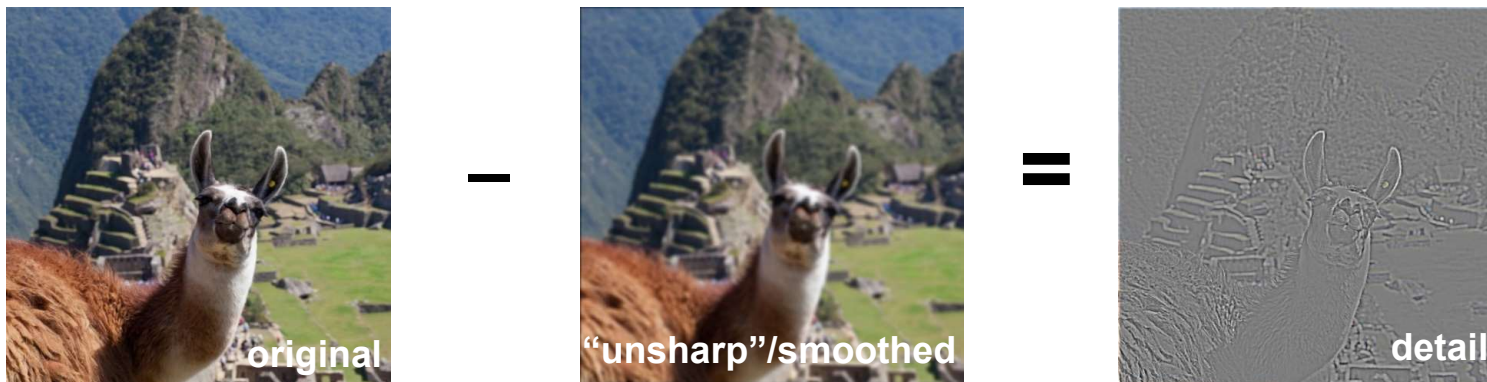
https://en.m.wikipedia.org/wiki/Unsharp_masking#/media/File%3AUnsharped_eye.jpg

Sharpening amplifies the high-frequency components of the image.

Unsharp Masking

- What does blurring take away?

Unsharp masking is an image sharpening technique



(This “detail extraction” operation is also called a *high-pass filter*)

Let's add it back:

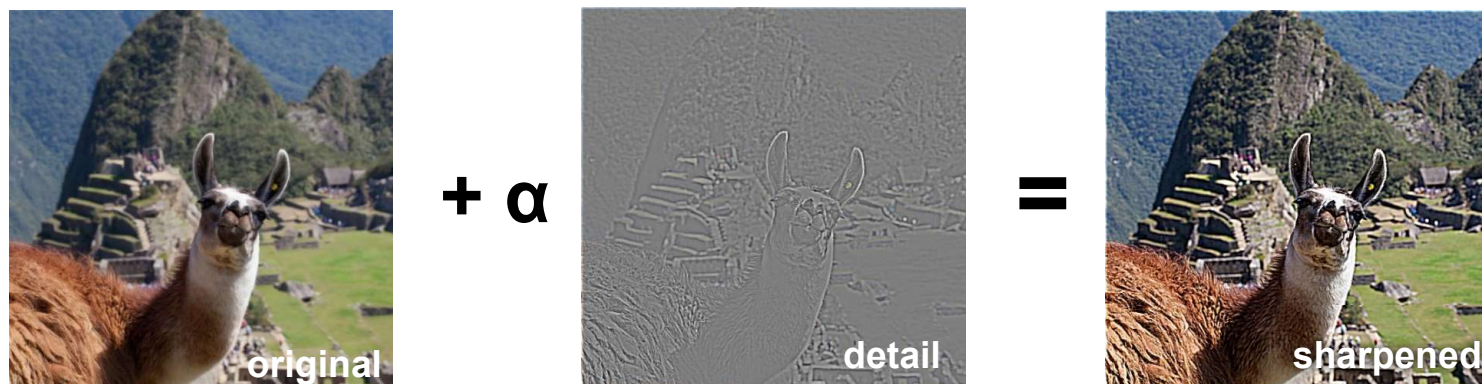
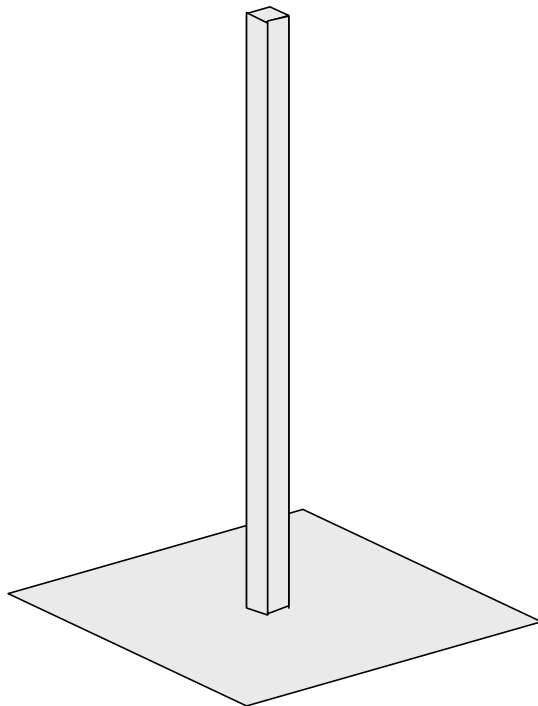


Photo credit: <https://www.flickr.com/photos/geezaweezer/16089096376/>

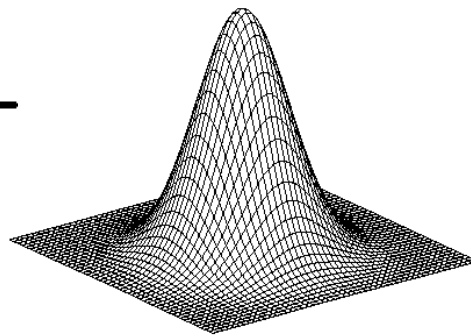
Unsharp Masking

$$\underset{\substack{\uparrow \\ \text{image}}}{F} + \alpha \left(F - \underbrace{F * H}_{\substack{\text{unsharp / blurred} \\ \text{image}}} \right) = (1 + \alpha) F - \alpha (F * H) = F * \left(\underset{\substack{\uparrow \\ \text{unit impulse} \\ \text{(identity kernel} \\ \text{with single 1 in} \\ \text{center, zeros} \\ \text{elsewhere)}}}{[1 + \alpha] e - \alpha H} \right)$$



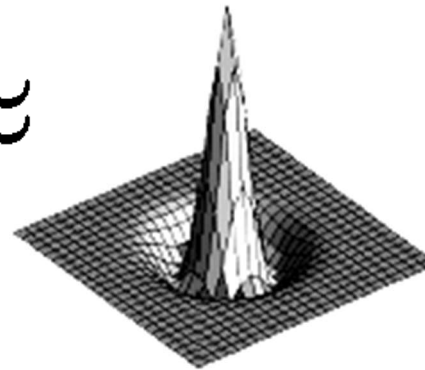
Scaled impulse

—



Scaled Gaussian

≈



Sharpen filter

unit impulse
(identity kernel
with single 1 in
center, zeros
elsewhere)

- Take-home message:** sharpening is taking an image F :
- 1) blurring it,
 - 2) subtracting the blurred image from F to get high-frequency details,
 - 3) then scaling and adding those edges back to F to yield an image with emphasized edges.

Unsharp Masking (technical note)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def unsharp_masking_gaussian_filter(image, gaussianFilter_size=3, amount=1.0):
    # If image has 3 channels, we can convert it to grayscale
    if len(image.shape) == 3:
        gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    else:
        gray_image = image

    # We apply Gaussian smoothing
    smoothed_image = cv2.GaussianBlur(gray_image, (gaussianFilter_size, gaussianFilter_size), 0)

    # We subtract the smoothed image to the original image to get the high-pass filtered image
    high_pass = cv2.subtract(gray_image, smoothed_image)

    # We add the details (high-pass) to the original image
    # dst = cv2.addWeighted(src1, alpha, src2, beta, gamma) ---> dst = alpha*src1 + beta*src2 + gamma
    sharp_image = cv2.addWeighted(gray_image, 1.0, high_pass, amount, 0.0)
    # = sharp_image = 1.0*gray_image + amount*high_pass + 0.0

    return gray_image, smoothed_image, high_pass, sharp_image

# Read image
image = cv2.imread('einstein.bmp', 0).astype(np.float64)

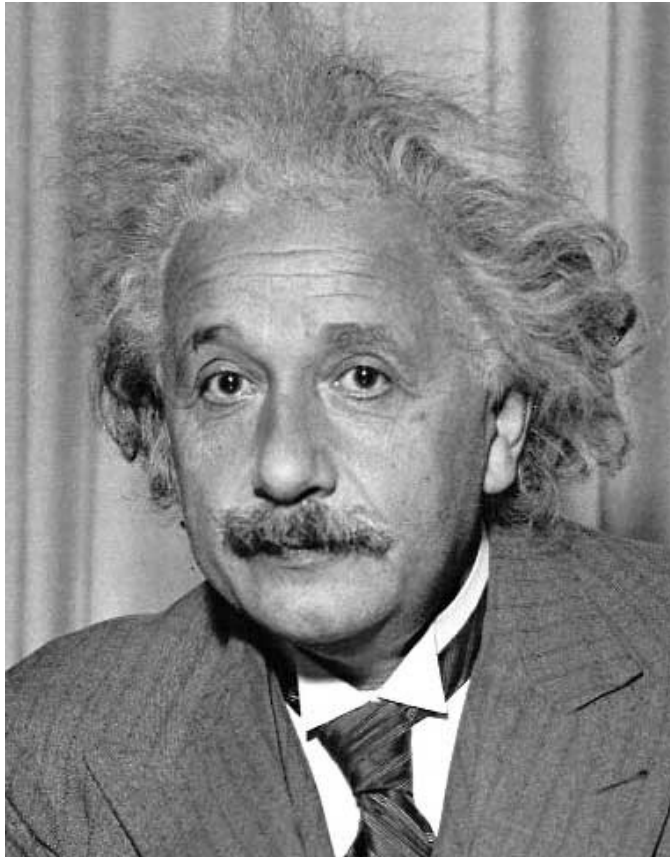
# Apply unsharp masking
gray_image, smoothed_image, high_pass, sharp_image = unsharp_masking_gaussian_filter(image, gaussianFilter_size=15, amount=1)
```

Unsharp Masking



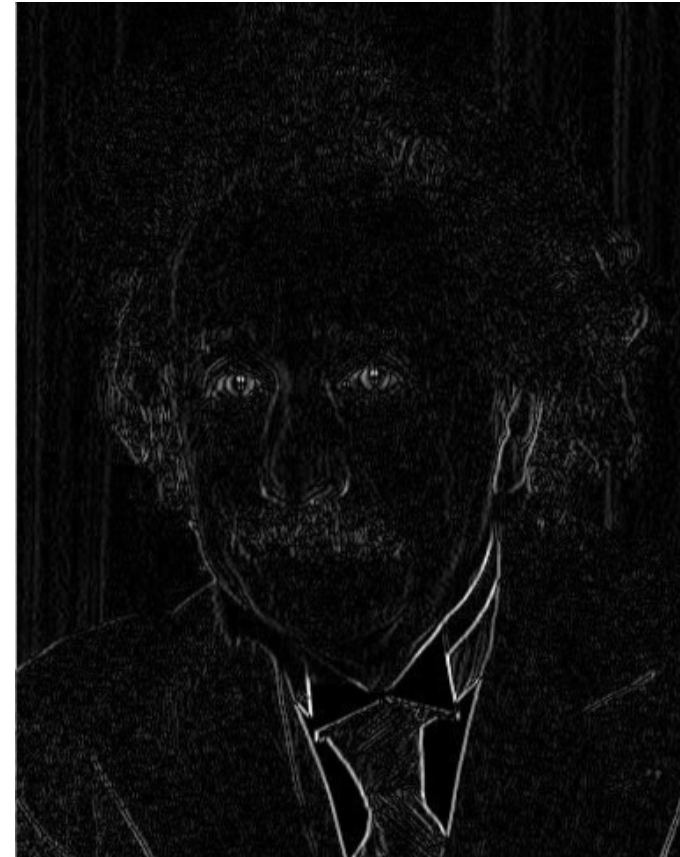
Edges are transformed so that the image gets darker on one side and brighter on the other – in this way, the edge is emphasized. **The sharpened image “anticipates” the edges**

Other filters



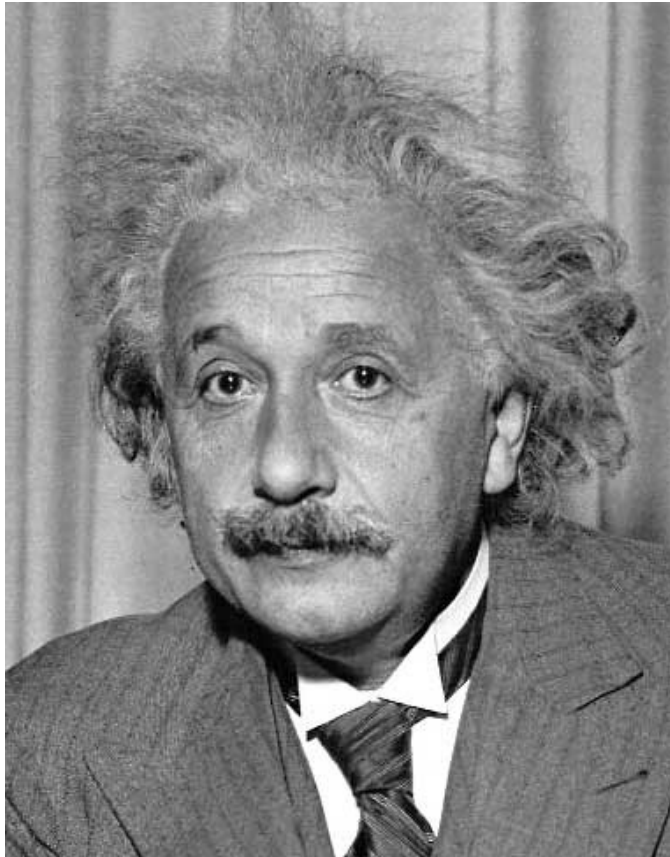
1	0	-1
2	0	-2
1	0	-1

Sobel



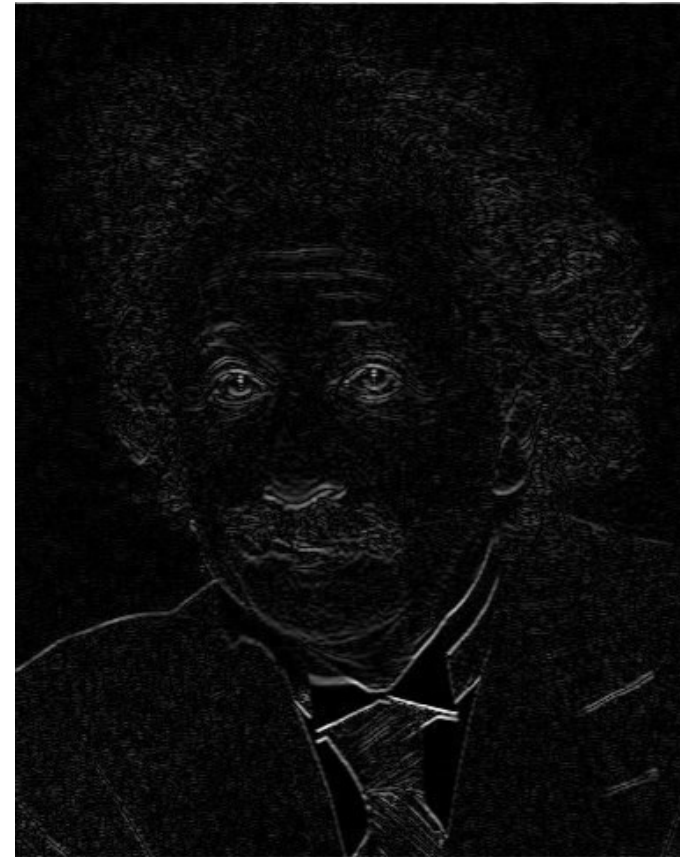
**Vertical Edge
(absolute value)**

Other filters



1	2	1
0	0	0
-1	-2	-1

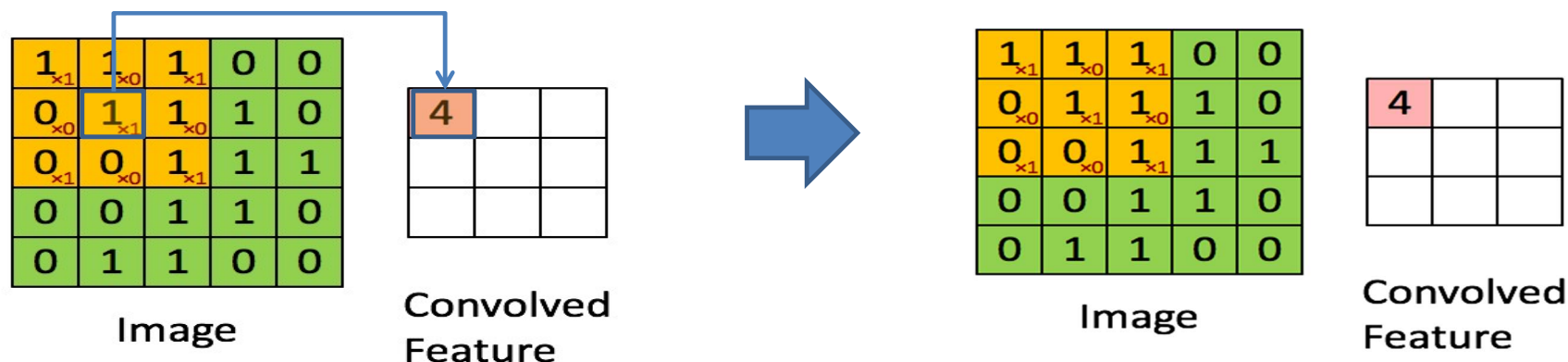
Sobel



**Horizontal Edge
(absolute value)**

Image Filtering (key ideas so far)

- Convolution is a **local and linear operation**
 - Red numbers below represent the coefficients of the mask.
 - The filter is multiplied element-by-element with the image, the products are added, and the central position of the filter is replaced in the image.



<https://glassboxmedicine.com/2019/07/26/convolution-vs-cross-correlation/>

Image Filtering (key ideas so far)

- Convolution is a **local and linear operation**
 - The coefficients of the mask determine the result

Gaussian filter (attenuates high frequencies → image smoothing)

$$\frac{1}{273}$$

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1



Sobel filter (attenuates low frequencies → enhances edges/boundaries)

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

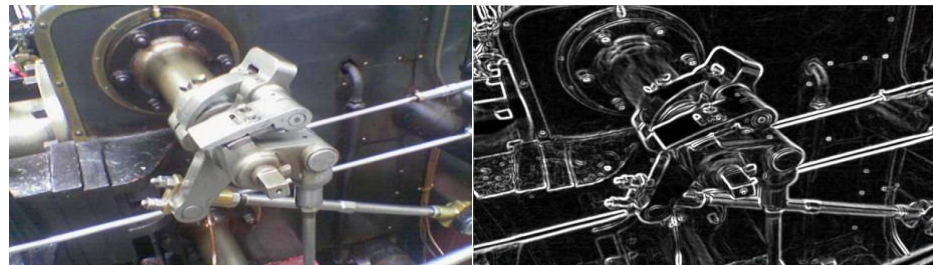


Image Filtering

- Image kernels visually explained
 - <https://setosa.io/ev/image-kernels/>

Let's walk through applying the following 3x3 **right sobel** kernel to the image of a face from above.

right sobel ▾

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Below, for each 3x3 block of pixels in the image on the left, we multiply each pixel by the corresponding entry of the kernel and then take the sum. That sum becomes a new pixel in the image on the right. Hover over a pixel on either image to see how its value is computed.

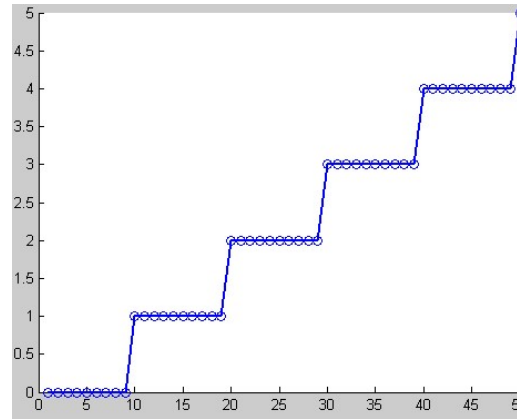


Nonlinear filters

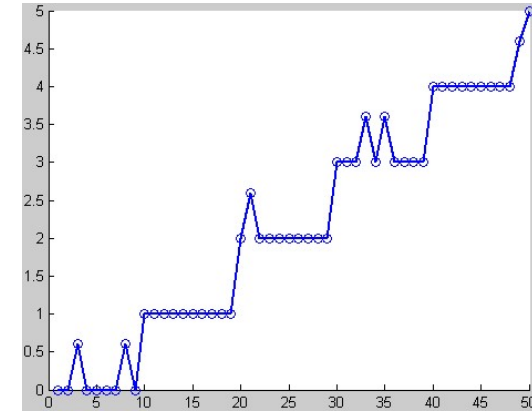
Median filter: Replace pixel with median of surrounding $n \times n$ region (no new pixel values introduced)

Good for impulse noise (salt-and-pepper noise)

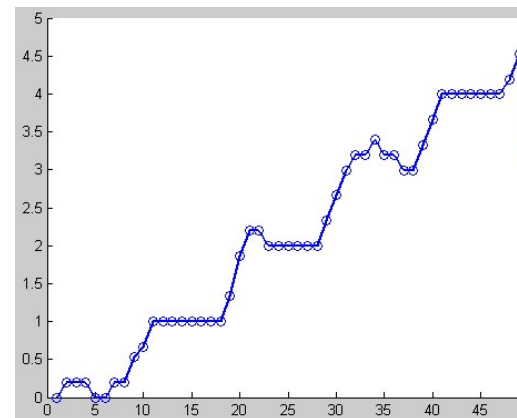
What is a mean filter good for?
Additive Gaussian noise



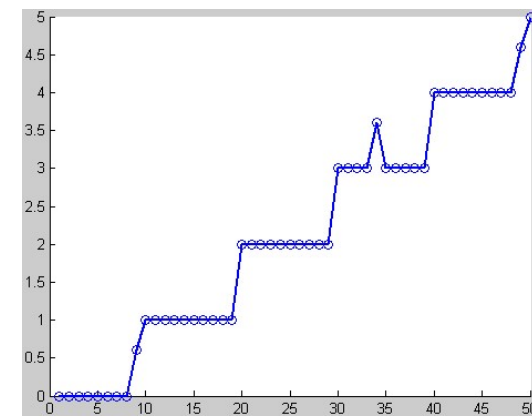
image



impulse noise



mean filtering



median filtering

Nonlinear filters

Image with salt-and-pepper noise



mean filtering (3x3)



mean filtering (11x11)



Why does a dark border appear
when using the 11×11 mean filter?

Nonlinear filters

Image with salt-and-pepper noise



median filtering (3x3)



median filtering (11x11)



Why does the dark border not appear when using the 11×11 median filter?

Thresholding



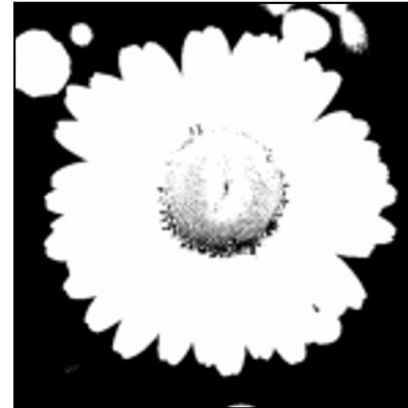
$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \text{otherwise} \end{cases}$$

Thresholding

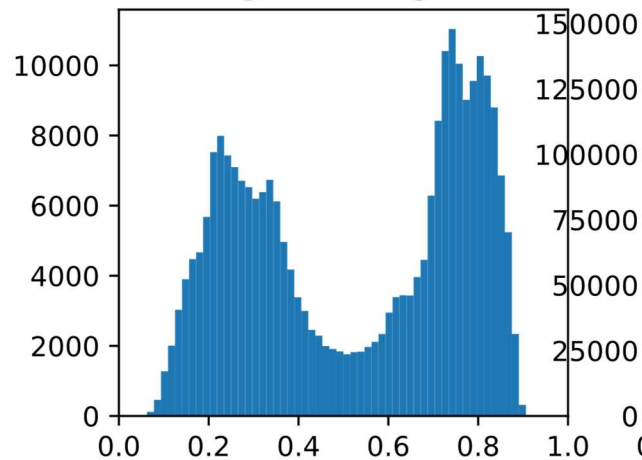
Original image



Transformed image



Original histogram



Transformed histogram

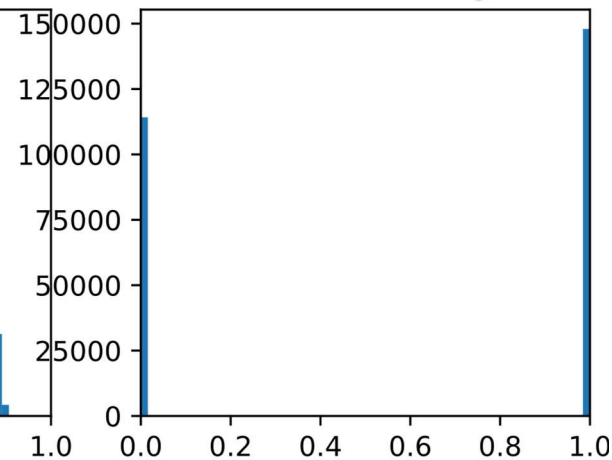


Image histogram
(number of pixels with a
particular gray-level value)

How to select this threshold? We can apply techniques like Otsu's method (Otsu, Nobuyuki. "A threshold selection method from gray-level histograms." *IEEE transactions on systems, man, and cybernetics* 9.1 (1979): 62-66)

<https://vincmazet.github.io/bip/histogram/transformations.html>

Otsu's Thresholding Method

We **iterate through all possible threshold values** and **measure the spread/variance** of background and foreground pixels. Then, **we select the threshold with the smallest spread/variance**. i.e. we want to minimize the within-class variance: $\sigma^2(t) = \omega_{bg}(t)\sigma_{bg}^2(t) + \omega_{fg}(t)\sigma_{fg}^2(t)$

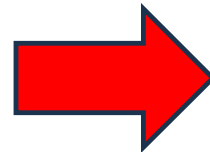
Let's try with this image and threshold (t=100):

Foreground Pixels

120	120	21	22
25	26	27	160
180	190	123	145
165	175	23	24

Background Pixels

$$\begin{aligned} P_{all} &= 16 \\ P_{BG} &= 7 \\ P_{FG} &= 9 \end{aligned}$$



$$\omega_{bg}(t) = \frac{P_{BG}(t)}{P_{all}} = 7/16 = 0.44$$

$$\omega_{fg}(t) = \frac{P_{FG}(t)}{P_{all}} = 9/16 = 0.56$$

$$\overline{x}_{bg} = \frac{21+22+25+26+27+23+24}{7} = 24$$

$$\overline{x}_{fg} = \frac{120+120+160+180+190+123+145+165+175}{9} = 153.1$$

$$\sigma_{bg}^2(t=100) = \frac{(21-24)^2 + (22-24)^2 + \dots + (24-24)^2}{7} = 4.0$$

$$\sigma_{fg}^2(t=100) = \frac{(120-153.1)^2 + (120-153.1)^2 + \dots + (175-153.1)^2}{9} = 657.43$$

$$\sigma^2(t=100) = 0.44 * 4.0 + 0.56 * 657.43 = 369.9208$$

We keep the threshold corresponding to the smallest σ^2

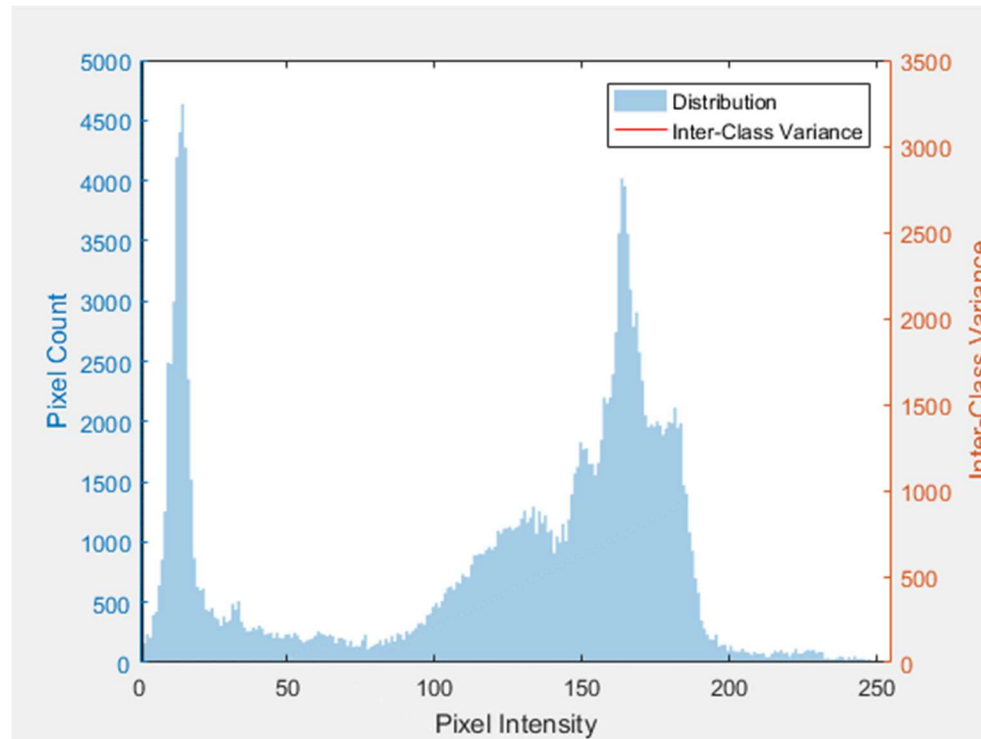
We want two clusters that are very cohesive internally (low variance)

Otsu, Nobuyuki. "A threshold selection method from gray-level histograms."
IEEE transactions on systems, man, and cybernetics 9.1 (1979): 62-66

<https://muthu.co/otsus-method-for-image-thresholding-explained-and-implemented/>

Otsu's Thresholding Method

Minimizing the intra-class variance is equivalent to maximizing inter-class variance



https://en.wikipedia.org/wiki/Otsu%27s_method#/media/File:Otsu's_Method_Visualization.gif



Otsu, Nobuyuki. "A threshold selection method from gray-level histograms."
IEEE transactions on systems, man, and cybernetics 9.1 (1979): 62-66

Check [Ejemplo_Otsu.ipynb](#)
in PRADO!

Histogram Equalization

We **increase the global contrast**, especially when the image is represented by a narrow range of intensity values.
After equalization, **the intensity values are more evenly distributed across the full range**.

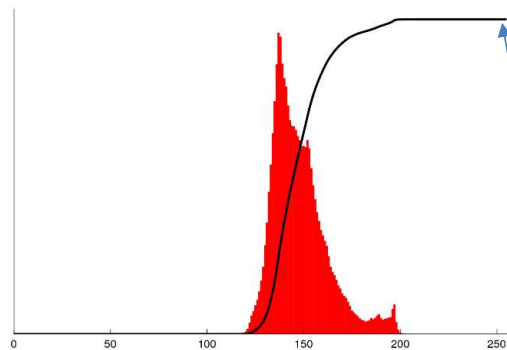


Before

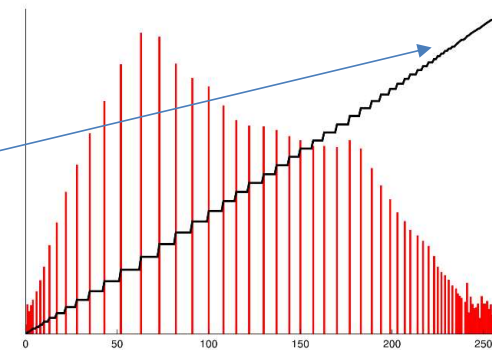
After



input intensity $r \rightarrow$ cumulative probability $CDF(r) \rightarrow$ output intensity $s = 255 \cdot CDF(r)$



$CDF(r)$ = fraction of pixels with intensity $\leq r$



Histogram (red) and cumulative histogram (black)

General Note

- In computer vision, our results (and our interpretation of them) are affected by two key aspects:
 - **Visualization** (i.e., we perform the correct operations, but our visualization makes us think that we are not doing the correct thing)
 - **Operation** (i.e., we are not performing the right operation (algorithmically or at the implementation level), but our visualization makes us think that we are doing the right thing)
 - This last situation is even more problematic, because we are doing something wrong without realizing it...
- We must be aware of these two dimensions in computer vision!

Homework about linear filters



- Is convolution equivariant to rotation and scaling?
- Is correlation equivalent to convolution if we rotate its output?
In other words, $rotate180(Correlation(I, K)) == Convolution(I, K)$?
- Can thresholding be implemented with a linear filter? In other words, is thresholding a linear operation?

Homework about linear filters



- Which one of the following 3x3 filters returns a positive value if the average value of the 4-adjacent neighbors is less than the center, and a negative value (or zero) otherwise?

(a)

0	$\frac{1}{4}$	0
$\frac{1}{4}$	-1	$\frac{1}{4}$
0	$\frac{1}{4}$	0

(b)

-1	-2	-1
0	0	0
1	2	2

(c)

0	-1	0
-1	4	-1
0	-1	0

Image Representation and Filtering

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



DaSCI

Instituto Andaluz de Investigación en
Data Science and Computational Intelligence